中图分类号： TP311 学校代码： 10055
UDC: 密级： 公开

南開大學

硕 士 学 位 论 文

弥合数字鸿沟：面向埃塞俄比亚工业园区的离线优先数字一站式服务
——本地化自中国智慧园区模式

Bridging the Digital Divide: An Offline-First Digital One-Stop Service Localized from China's Smart Park Model for Ethiopia's Industrial Parks

论文作者 Kebede Yetnayet Berhanu 指导教师 张海宁 教授

申请学位 工学硕士 培养单位 软件学院

学科专业 软件工程 研究方向 数字平台开发

答辩委员会主席 _____ 评阅人 _____

南开大学研究生院

二〇二五年十二月

南开大学学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下进行研究工作所取得的研究成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或者没有公开发表的作品的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律责任由本人承担。

学位论文作者签名：_____ 2025 年 12 月 日

非公开学位论文标注说明

(本页表中填写内容须打印)

根据南开大学有关规定，非公开学位论文须经指导教师同意、作者本人申请和相关部门批准方能标注。**未经批准的均为公开学位论文，公开学位论文本说明为空白。**

论文题目			
申请密级	□限制 (⩽2 年)	□秘密 (⩽10 年)	□机密 (⩽20 年)
保密期限	20 年 月 日至 20 年 月 日		
审批表编号		**批准日期**	20 年 月 日

南开大学学位评定委员会办公室盖章 (有效)

注：限制★2 年 (可少于 2 年) ; 秘密★10 年 (可少于 10 年) ; 机密★20 年 (可少于 20 年)

南开大学学位论文使用授权书

本人完全了解《南开大学关于研究生学位论文收藏和利用管理办法》关于南开大学 (简称“学校”) 研究生学位论文收藏和利用的管理规定，即：

南开大学拥有在著作权法规定范围内学位论文的使用权，其中包括：

已获学位的研究生必须按学校规定提交学位论文，学校可以采用影印、缩印或其他复制手段保存学位论文；学校根据规定向教育部指定的收藏和存档单位提交学位论文；

为教学和科研目的，学校可以将公开的学位论文作为资料在图书馆、资料室等场所供校内师生阅读，或在校园网范围内供师生检索、浏览；

非公开学位论文在解密后的使用权同公开论文。

本人承诺：本人的学位论文是在南开大学学习期间创作完成的作品，并已通过论文答辩；提交的学位论文电子版与纸质本论文的内容一致，如因不同造成不良后果由本人自负。

本人签署本授权书一份（此授权书为论文中一页），交图书馆留存。

学位论文作者暨授权人 (亲笔) 签字：

2025 年 12 月 日

南开大学研究生学位论文作者信息

论 文 题 目	弥合数字鸿沟：面向埃塞俄比亚工业园区的离线优先数字一站式服务				
姓 名	Kebede Yetnayet Berhanu	学号	2120246030	答 辩 日 期	
论 文 类 别	博士☐ 学历硕士☑ 专业学位硕士☐ 同等学力硕士☐ 划☑选择				
学院 (单位)	软件学院		学科 / 专业 (专业学位) 名称	软件工程	
联 系 电 话			电子邮箱		
通讯地址 (邮编)：天津经济开发区宏达街 23 号南开大学泰达校区（300457）					
非 公 开 论 文 编 号		备注			

注：本授权书适用我校授予的所有博士、硕士的学位论文。如已批准为非公开学位论文，须向图书馆提供批准通过的《南开大学研究生申请非公开学位论文审批表》复印件和“非公开学位论文标注说明”页原件。

摘要

埃塞俄比亚工业园区发展公司（IPDC）管辖的多个工业园区目前仍依赖纸质行政流程，导致服务交付延迟、费用收缴效率低下，并引发利益相关方的不满。在中国，基于企业微信和钉钉构建的智慧园区平台已在数字化治理方面取得显著成效，但这些平台依赖稳定的互联网连接——而埃塞俄比亚工业园区无法保证这一条件，日均网络中断时间长达三至五小时。

本研究旨在为 IPDC 设计、开发并评估一个离线优先的数字一站式服务平台，以渐进式网络应用程序（PWA）形式交付。研究采用设计科学研究方法论，将中国智慧园区模式适配到网络受限环境中。三个研究问题贯穿全文：（1）如何系统地将中国数字治理模式迁移到非洲环境；（2）如何架构一个支持复杂工业工作流的离线优先 PWA；（3）在服务多个利益相关方群体的平台中，应以何种设计原则指导功能优先级决策。

所开发的原型系统支持离线服务请求提交并在网络恢复后自动同步，提供基于角色的差异化界面，以及用于跟踪服务费用的数字令牌系统。由于采用 PWA 技术构建，该平台可通过单一代码库在桌面端和移动端浏览器上运行，无需开发独立的原生移动应用。Firebase Firestore 作为后端服务，提供内置的离线数据持久化能力。系统还集成了三个机器学习模型：自动服务分类模型、预测性设备维护模型以及面向潜在租户的工业园区推荐模型。

本研究做出四项贡献：（1）提出技术适配框架，指导数字治理模式在不同网络条件下的跨境迁移；（2）设计离线优先工业服务架构，扩展 PWA 能力以支持复杂工作流；（3）建立一套设计原则，用于确定离线使用场景下的功能优先级；（4）开发首个面向埃塞俄比亚工业园区的专用数字平台。这些成果为非洲及其他发展中地区在网络受限环境下的数字化转型提供了可复制的方法。

关键词： 离线优先架构，渐进式网络应用，数字一站式服务，智慧工业园区，设计科学研究，技术适配，埃塞俄比亚工业园区

Abstract

IPDC — Ethiopia's Industrial Parks Development Corporation — runs several parks where service requests still travel on paper. Tenants visit offices in person, fees get collected manually, and when a request stalls there is usually no way to know why. Chinese smart park operators solved exactly this problem years ago using platforms like WeChat Work and DingTalk, but those platforms were built for reliable broadband. Ethiopian parks lose internet for three to five hours on a typical day. Importing the Chinese solution without rethinking its architecture would simply trade one failure mode for another.

This research designed, built, and evaluated an offline-first digital one-stop service for IPDC, delivered as a Progressive Web Application. Design Science Research guided the process because the goal was a working artifact, not just an analysis of the problem. Three questions shaped the work: how Chinese digital governance models can be adapted for African infrastructure realities; what architecture lets a PWA handle complex industrial workflows without a live internet connection; and which design principles should determine what works offline versus what can wait for connectivity.

The resulting prototype supports service request submission while offline, syncing automatically when the network returns. Three stakeholder roles — management, tenants, and operations staff — each get purpose-built interfaces. A token system handles service fee tracking digitally. One codebase runs on desktop and mobile browsers alike, removing the need to build separate native apps. Firestore provides backend data persistence with offline support built in. Three machine learning models handle service classification, equipment maintenance prediction, and park recommendation for prospective tenants.

The research produced four outputs: a Technology Adaptation Framework for transferring digital governance models across connectivity contexts; an offline-first architecture suited to multi-stakeholder industrial workflows; a set of design principles for prioritizing features under variable connectivity; and the platform prototype itself

— the first digital one-stop service built specifically for Ethiopian industrial parks. The approach is not unique to Ethiopia. Any organization in a similar infrastructure position can take the same starting point and adapt it.

Key Words: Offline-First Architecture, Progressive Web Application, Digital One-Stop Service, Smart Industrial Park, Design Science Research, Technology Adaptation, Ethiopian Industrial Parks

CONTENTS

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
BaaS	Backend as a Service
BRI	Belt and Road Initiative
CBE	Commercial Bank of Ethiopia
CORS	Cross-Origin Resource Sharing
CRDT	Conflict-free Replicated Data Type
CSS	Cascading Style Sheets
DSR	Design Science Research
DSRM	Design Science Research Methodology
ETB	Ethiopian Birr
HTML	HyperText Markup Language
ICT	Information and Communication Technology
ICT4D	ICT for Development
IPDC	Industrial Parks Development Corporation
JS	JavaScript
MoSCoW	Must Have, Should Have, Could Have, Won't Have
MUI	Material-UI
NoSQL	Not Only SQL
OSS	One-Stop Service
PIM	Park Innovation Management
PWA	Progressive Web Application
RBAC	Role-Based Access Control
SDK	Software Development Kit
SEZ	Special Economic Zone
SNNPR	Southern Nations, Nationalities, and Peoples' Region
SQL	Structured Query Language
SUS	System Usability Scale
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines

Chapter 1 Introduction

1.1 Background of the Study

1.1.1 Global Context: Digital Transformation in Industrial Parks

Industrial parks used to offer land, utilities, and little else. That's changed. Digital platforms now run a significant part of park administration in many parts of the world — service requests, tenant communication, operational data — and parks without them are increasingly the exception rather than the norm [36, 74].

China's experience here is worth looking at closely. Parks built around WeChat Work or DingTalk have brought business registration, permit processing, facility management, and tenant communication onto one screen [36, 60]. It isn't simply faster paperwork. The nature of the relationship between park managers and the companies they host has genuinely changed.

Developing economies still running industrial zones on manual administration have noticed, and for obvious reasons. The Belt and Road Initiative, alongside the Digital Silk Road, has opened working channels for this kind of transfer between China and African partners [21, 40, 44]. Whether a model built for Shenzhen can work in Hawassa is a different question entirely — but the interest is real.

1.1.2 Ethiopian Context: Industrial Parks and Digital Transformation

Industrial parks are central to Ethiopia's economic strategy, and the Industrial Parks Development Corporation (IPDC) — established in 2014 — is the institution responsible for building and managing them [37, 54]. Parks at Bole Lemi, Hawassa, Eastern Industrial Zone, and elsewhere host manufacturers from Ethiopia and abroad; their exports and employment figures feed directly into national development plans [32, 65].

Chinese involvement goes well beyond trade flows. Chinese capital and expertise built several of the parks, and Chinese companies remain a substantial share of the

tenant base. That pre-existing relationship makes the question worth asking: could the digital governance tools that Chinese parks have developed actually be made to work in Ethiopia? [50].

Right now, the honest answer is that the current setup isn't working well. Requesting a service means visiting an office. Once submitted, a request disappears into a manual process and there's no way to track it. Fee payments are separate — cash, or a bank branch trip — with nothing linking them back to the request that triggered them. Firms arriving from China or India, where this is all handled on a phone, find the contrast jarring, and research confirms it shapes how they assess whether to stay [30, 31].

1.1.3 The Connectivity Challenge: Infrastructure Constraints

Any attempt to build a digital platform for Ethiopian industrial parks hits the same wall quickly: the internet goes out. Survey respondents and national statistics [22, 38] describe the same situation — parks typically lose connectivity for three to five hours per day, with full outages two to three times a month. Chinese parks run on stable high-speed fibre. Ethiopian parks don't.

Investment in infrastructure is happening [72], but it's slow, and the near-term picture isn't dramatically different from today. A cloud-dependent platform would go offline every time the connection drops — arguably worse than the paper system it's meant to replace, since paper at least doesn't need a fibre link to function.

WeChat Work and DingTalk weren't designed for this. They assume connectivity is there. Drop them into Ethiopian conditions unchanged and you get a system that breaks whenever it's most needed, which is precisely when people would give up on it and go back to manual processes. The problem isn't at the surface level — it's architectural.

1.1.4 The Offline-First Paradigm: A Solution Framework

Offline-first thinking inverts the usual design assumption. Instead of treating network access as the default and outages as an exception to handle, it makes local operation the baseline. The network is useful when it's there — for syncing — but the application doesn't need it to function. Kleppmann et al. [39] laid this out in their local-

first software work, and it's a direct fit for the Ethiopian connectivity situation.

Progressive Web Applications turn this into something practical. Service workers intercept requests and serve cached responses when the network is unavailable; IndexedDB holds structured data on the device; background sync ships queued operations to the server once the connection is back [5, 45, 63]. Research comparing PWAs and native apps finds comparable user experience results, and a single PWA codebase covers both desktop and mobile without building twice [1, 11].

Firebase works well with this. Firestore's offline persistence caches data automatically and queues writes for later upload — the application doesn't have to manage any of that transition [27]. The free tier covers 50,000 daily reads, 20,000 writes, and 1 GB of storage, which is more than enough for a prototype and an initial pilot. Put Chinese smart park governance thinking, offline-first PWA design, and Firebase's built-in persistence together, and you have a foundation for park administration that doesn't fall over when the internet does.

1.2 Statement of the Problem

IPDC manages industrial parks serving Ethiopian and international manufacturers. Park administration covers a wide range — business licensing, facility management, utility services, maintenance coordination, regulatory compliance — and how well it works directly shapes whether tenants stay, expand, or leave. Right now, it doesn't work well, and the reasons aren't complicated.

The whole system runs on paper and informal communication. Tenants file requests in person. They track progress by calling or visiting again. The process is opaque by design, or at least by neglect. Several specific problems follow:

Service Delivery Inefficiency: Paper workflows slow request processing and approval routing. Requests get lost, misdirected, or stalled when staff are unavailable. Without automated tracking, neither staff nor tenants can reliably tell where any request stands.

Fee Collection Challenges: Fee collection is fragmented and lacks transparency. Cash payments raise security concerns; bank transfers require separate branch visits; and reconciling payments with services is a manual, error-prone process with no inte-

grated system linking requests to payment status.

Communication Gaps: Information flows through multiple informal channels between management, administrative staff, operations teams, and tenants. Important announcements may not reach everyone, and there is no structured mechanism for tenant feedback.

Operational Opacity: Without digital records, management cannot answer basic operational questions: How long does a typical request take to resolve? Which service categories generate the most complaints? Where are staffing bottlenecks occurring? These questions are fundamental to running a well-managed park, yet there is currently no systematic way to answer them.

Stakeholder Frustration: Ethiopian industrial parks compete for international investment. Tenant firms arriving from China, India, or other countries where park administration is already digital encounter a jarring step backward at IPDC. That experience shapes retention decisions and the attractiveness of IPDC parks to prospective investors who can compare alternatives.

Going digital is the logical response, but a standard cloud-based system would simply create a new version of the same problem. An internet-dependent platform goes dark during every outage — and given that Ethiopian parks lose connectivity for hours at a time, that means the system fails exactly when staff are trying to process requests and tenants are waiting for updates. The literature does not offer much help here: Chinese smart park research assumes stable broadband [70, 73]; offline-first and PWA studies tend to focus on simpler single-user citizen services rather than multi-party industrial workflows [17, 49]; and e-government work in developing countries acknowledges connectivity constraints without providing concrete architectural solutions for industrial settings [18, 42].

This research addresses that gap directly by designing and building an offline-first digital one-stop service that adapts Chinese smart park governance principles for the Ethiopian context. The central question driving the work is: *How can Chinese smart park digital governance models be systematically adapted using offline-first PWA architecture to deliver reliable digital services in connectivity-constrained Ethiopian industrial parks?*

1.3 Motivations

A few specific things motivated this work beyond the general problem:

- **Operational inefficiency that has a known fix.** Paper workflows create delays, lost requests, and frustrated tenants — and a digital platform that keeps working during outages would address this concretely, not theoretically.
- **A design constraint the literature ignores.** Most digital governance tools assume reliable internet. Ethiopian parks lose connectivity for three to five hours daily. Treating that as a design requirement rather than an obstacle to work around is the starting point this research takes.
- **An existing China-Ethiopia relationship to build on.** Chinese firms built several Ethiopian parks and still make up a large share of tenants. There are tested digital governance platforms on the Chinese side, and there is no systematic framework for adapting them for African infrastructure conditions. Developing that framework is one reason this work exists.
- **Scalability across the IPDC portfolio.** A single PWA codebase can serve every IPDC park — desktop and mobile — without duplicating development. That makes a working prototype at one park a realistic starting point for nationwide deployment.
- **No digital records means no management data.** Paper systems produce nothing useful for analytics. Integrating AI capabilities for service classification, maintenance prediction, and park recommendation would give IPDC management tools they currently have no access to.
- **An untested application of offline-first principles.** PWA offline research typically covers simple citizen-facing services. Multi-stakeholder industrial workflows with approvals, token transactions, and complaint handling are a different problem. Working through it here produces patterns others can reuse.

1.4 Research Questions

Three research questions structure the study. Each addresses a different part of the problem.

RQ1: What systematic adaptation methodology enables the transfer of Chinese Smart Park digital governance models to connectivity-constrained African industrial parks?

This question addresses the conceptual challenge of taking systems designed for high-connectivity settings and adapting them for infrastructure-limited environments. It examines which elements of Chinese models can be transferred, what changes are necessary, and what approach should guide such adaptations.

RQ2: How should offline-first PWA architecture be designed to maintain functional parity with online systems for complex industrial park service workflows?

Keeping industrial park services fully functional during connectivity disruptions is the technical core of this question — caching strategies, synchronization, conflict resolution, and offline user experience all feed into the answer.

RQ3: What design principles should guide feature prioritization for offline availability in multi-stakeholder industrial platforms?

Not every feature can be equally critical or technically feasible for offline use. This question explores prioritization approaches that balance stakeholder needs with technical constraints.

1.5 Research Objectives

1.5.1 General Objective

The overall objective of this research is to design, develop, and evaluate an offline-first digital one-stop service Progressive Web Application (PWA) for the Ethiopian Industrial Parks Development Corporation (IPDC) by systematically adapting Chinese smart park digital governance models using Firebase backend services for connectivity-constrained environments.

1.5.2 Specific Objectives

To achieve this overall objective, the research pursues the following specific objectives:

1. To analyze and document existing service delivery processes and stakeholder

requirements across IPDC industrial parks through comprehensive requirement gathering involving tenant firms, administrative staff, and operations personnel.

2. To develop a systematic adaptation framework identifying which elements of Chinese smart park models (WeChat Work, DingTalk) can be transferred and what modifications are needed for Ethiopian infrastructure constraints.
3. To design an offline-first system architecture incorporating PWA technologies (service workers, IndexedDB, background sync) together with Firebase backend services (Firestore with offline persistence, Authentication, Cloud Functions).
4. To build a functional PWA prototype accessible through web browsers on both desktop and mobile devices, demonstrating offline service request submission, digital service token-based fee tracking, local data persistence, automated synchronization, and role-based interfaces.
5. To evaluate the prototype through technical testing (offline functionality, synchronization accuracy, performance) and usability assessment with representative stakeholders.
6. To derive and document design principles for offline-first industrial service platforms based on the development experience and evaluation findings.

Table 1.1 maps the specific objectives to research questions and expected deliverables.

Table 1.1 Mapping of Objectives, Research Questions, and Deliverables

Specific Objective	RQ	Expected Deliverable
1. Analyze requirements	RQ1, RQ3	Requirements specification with stakeholder analysis
2. Develop adaptation framework	RQ1	China-to-Ethiopia adaptation framework
3. Design offline-first architecture	RQ2	System architecture with PWA and Firebase design
4. Implement PWA prototype	RQ2, RQ3	Functional PWA accessible on desktop and mobile browsers
5. Evaluate prototype	RQ2, RQ3	Evaluation report with technical and usability findings
6. Derive design principles	RQ1, RQ2, RQ3	Design principles for offline-first industrial platforms

1.6 Significance of the Study

The research makes contributions at three levels. **Academically**: the China-to-Ethiopia Technology Adaptation Framework fills a gap no prior work has addressed; the offline-first architecture extends PWA research beyond simple citizen services into complex multi-stakeholder industrial workflows; and the DSR application to an African industrial setting demonstrates that research rigor and practical relevance can coexist in that context. **Practically**: the prototype gives IPDC a working starting point for digital transformation; the design principles and architecture patterns can serve as a template for similar projects across Africa; and the single-codebase PWA approach cuts development cost for organizations with limited resources. **For specific stakeholders**: tenants get service access that works during outages with transparent status tracking; administrative staff get digital workflows and an audit trail replacing paper; operations personnel can update task assignments in the field without connectivity; and IPDC management gets operational data — volumes, resolution times, resource utilization — they currently have no way to access.

1.7 Scope and Limitations

1.7.1 Scope of the Research

Geographic Scope: The research targets IPDC-managed industrial parks in Ethiopia, drawing primary data from Bole Lemi and Hawassa Industrial Parks —two parks that differ in location, size, and tenant composition, providing a range of perspectives.

Stakeholder Scope: Three groups are involved: tenant firm representatives, IPDC administrative staff, and operations personnel.

Functional Scope: The prototype covers core service delivery functions: service request submission and tracking, maintenance request management, announcements, document management, and digital token-based fee tracking.

Technical Scope: The research produces a PWA built with modern web technologies, delivering cross-platform accessibility across desktops, tablets, and phones from a single codebase, with home screen installation and offline access. Firebase provides the cloud backend via its built-in Firestore offline persistence.

Fee Tracking Scope—Digital Service Token System: Rather than integrating with external payment gateways, the platform uses an internal token system: administrators assign tokens to tenant accounts, fees are deducted on request, and tokens are topped up when external payments are confirmed. This demonstrates the complete fee workflow with a clear path to future payment gateway integration.

Scope Exclusions: The research leaves out native mobile app development, external payment gateway integration, IoT sensor integration, and advanced analytics — all noted as directions for future work.

1.7.2 Limitations

Prototype Stage: The research produces a functional prototype rather than a production-ready system. Moving to full deployment would require additional security hardening and scalability work.

Internal Token System: While the token approach demonstrates the full fee management workflow, it requires manual administrator action when external payments come in. Integration with services like telebirr or CBE Birr is a future enhancement

once the necessary API partnerships are in place.

Evaluation Constraints: The prototype is evaluated with a representative stakeholder group rather than through comprehensive testing across all parks.

Connectivity Simulation: Offline functionality testing necessarily uses simulated offline conditions rather than real network outages.

Language Considerations: The interface is in English, the main business language in Ethiopian industrial parks. Amharic localisation is noted as a future enhancement.

1.8 Theoretical Framework

Four theoretical threads run through this research, each shaping a different aspect of the design and analysis.

1.8.1 Design Science Research (DSR)

Design Science Research serves as the overarching methodological framework. Following Hevner et al. [35] and Peffers et al. [57], this research creates a purposeful IT artifact — the offline-first digital platform — while also generating design knowledge. The work adheres to Hevner's seven DSR guidelines: design as artifact, problem relevance, design evaluation, research contributions, research rigor, design as search process, and communication of research.

1.8.2 Appropriate Technology Theory

Rooted in Schumacher [61] and updated for the digital age by Toyama [66], Appropriate Technology Theory insists that solutions must fit local conditions —infrastructure capabilities, available skills, economic resources, and existing ecosystems. The decision to use an internal token system rather than requiring external payment gateway partnerships is one concrete expression of this principle: it produces a self-contained solution that works within current organisational capabilities while remaining extensible.

1.8.3 Local-First Software Principles

As described by Kleppmann et al. [39], local-first principles put local data storage and processing first, treating network connectivity as an enhancement rather than a requirement. These principles directly shape key architecture decisions: using IndexedDB for client-side persistence, implementing service workers for request caching, and designing background synchronization strategies. Firebase Firestore's built-in offline persistence aligns naturally with this philosophy.

1.8.4 Technology Transfer Framework

Drawing on South-South technology transfer literature [16, 34], the research treats technology adaptation as an active, context-sensitive process rather than simple replication. The framework accounts for technological, organisational, cultural, and economic dimensions of the China-to-Ethiopia transfer.

1.9 Definition of Key Terms

Offline-First Architecture: A software design approach treating local data storage as primary, so the application remains fully functional without network access; connectivity enhances synchronization but is not a prerequisite.

Progressive Web Application (PWA): A web application using service workers, manifests, and caching APIs to deliver app-like offline capability, push notifications, and home screen installation across desktop and mobile browsers.

One-Stop Service (OSS): A service delivery model consolidating multiple services into a single access point, eliminating the need to visit separate channels or locations.

Digital Service Token: An internal credit unit for tracking service fees; administrators assign tokens to tenant accounts and the system deducts them when services are requested.

Firebase: Google's Backend-as-a-Service platform offering Firestore (cloud database with offline persistence), authentication, hosting, and cloud functions.

Service Worker: A JavaScript file running in a background browser thread that enables request interception, caching, and offline functionality.

IndexedDB: A browser API for client-side structured data storage, used here via Dexie.js as the local offline queue and data cache.

Smart Park: An industrial park using digital technologies for intelligent management, automated services, and data-driven decision-making.

Digital Governance: The use of digital technologies to deliver organisational services, engage stakeholders, and manage operations electronically.

Design Science Research (DSR): A research paradigm centered on creating and evaluating IT artifacts to solve identified organisational problems, producing practical solutions and generalizable knowledge.

Industrial Parks Development Corporation (IPDC): The Ethiopian government corporation, established in 2014, responsible for developing, managing, and operating industrial parks across Ethiopia.

1.10 Thesis Organization

This thesis consists of six chapters following the Design Science Research Methodology (DSRM) process model. Table 1.2 shows how DSRM activities map onto chapters.

Chapter One: Introduction establishes the research context, problem statement, research questions, objectives, significance, scope, theoretical framework, and key definitions.

Chapter Two: Literature Review surveys Chinese smart park ecosystems, the BRI and Digital Silk Road, digital governance in developing countries, offline-first and PWA technologies, ICT4D, technology adaptation frameworks, the Ethiopian industrial parks context, and DSR methodology —concluding with a gap analysis.

Chapter Three: Research Methodology describes the DSR approach, data collection methods, development methodology, and evaluation framework.

Chapter Four: System Analysis and Design presents the requirements analysis, the China-to-Ethiopia adaptation framework, and the system architecture.

Chapter Five: Implementation and Evaluation covers prototype development, demonstrates key features, and reports technical testing and usability assessment findings.

Chapter Six: Conclusions and Recommendations synthesizes findings, presents design principles, discusses implications, acknowledges limitations, and suggests future work.

Table 1.2 Mapping of DSRM Activities to Thesis Chapters

DSRM Activity	**Thesis Chapter**	**Key Outputs**
Problem Identification & Motivation	Chapter 1, Chapter 2	Problem statement, research gap
Define Objectives of a Solution	Chapter 1, Chapter 3	Objectives, requirements
Design & Development	Chapter 4, Chapter 5	Architecture, prototype
Demonstration	Chapter 5	Use cases, screenshots
Evaluation	Chapter 5, Chapter 6	Testing results, usability findings
Communication	Chapter 6	Design principles, thesis document

Chapter 2 Literature Review

2.1 Introduction

Four bodies of literature shaped this research in different ways. Chinese smart park work supplied the design model. Offline-first and PWA research gave the technical approach. ICT4D and e-government scholarship set the development context. And Design Science Research structured the methodology. Each is reviewed in turn; the chapter closes with a gap analysis.

2.2 Chinese Smart Park Digital Ecosystems

2.2.1 Evolution of Smart Industrial Parks in China

Chinese industrial parks have changed substantially over forty years. Yang, Zhou and Li [74] identify three phases: an establishment period through the 1990s that prioritised attracting foreign capital; an optimisation period in the 2000s and 2010s focused on industrial clustering; and a smart phase from around 2015 onward defined by digital transformation [15, 75, 76].

The shift is not just technological. Huang and Zhang [36] describe it as a change in how parks understand what they are for — moving from "space provider" to "service ecosystem." Parks that once delivered land and utilities now manage tenant operations through digital platforms.

In practice this plays out as stacked technology layers: IoT sensors monitoring the physical environment, digital platforms handling administrative services, analytics tools helping management make sense of operational data, and connections to national e-government systems [70]. It's not a standardised architecture — each park does it somewhat differently — but the underlying logic is similar across implementations.

2.2.2 Digital One-Stop Service Platforms

What distinguishes Chinese smart park platforms from earlier arrangements is the one-stop model. Instead of directing tenants to separate departments, parks built integrated interfaces that handle everything in one place — borrowed from e-government reform and adapted for the industrial park context [73].

Schreieck, Wiesche and Krcmar [60] examine platform strategies in Chinese industrial parks and identify three common approaches: proprietary platform development, adoption of commercial enterprise platforms (WeChat Work, DingTalk), and hybrid models combining aspects of both. Parks using commercial platforms benefit from established user familiarity and mature feature sets; proprietary platforms, meanwhile, offer greater customization.

WeChat Work (internationally known as WeCom) has emerged as a dominant vehicle for smart park digital services. As of 2021, Tencent [64] report over 5.5 million enterprise users, with industrial parks forming a notable segment. The platform provides enterprise messaging, workflow automation, approval routing, and integration with the broader WeChat ecosystem — including payment services. DingTalk, Alibaba's alternative, offers comparable functionality with particular strengths in workflow management and cloud integration. Alibaba Group [2] document features such as OA approval workflows, collaborative document editing, attendance tracking, and video conferencing for up to 1,000 participants, with low-code tools enabling rapid customization.

2.2.3 Park Innovation Management Framework

Yang, Zhou and Li [74] propose the Park Innovation Management (PIM) framework as a lens for understanding digital transformation in Chinese industrial parks. Five interconnected dimensions structure the framework: institutional innovation (governance structures and policies), technological innovation (digital infrastructure and platforms), service innovation (new service delivery models), ecosystem innovation (stakeholder relationships and value networks), and spatial innovation (bridging the physical and digital).

For this research, the PIM framework is useful because it maps which elements of Chinese smart park models might transfer and which will require rethinking. More

importantly, it shows that technology transfer cannot work by copying the software alone — the institutional, cultural, and service innovation dimensions matter just as much.

2.2.4 Suzhou Industrial Park Case Study

Suzhou Industrial Park started in 1994 as a China-Singapore collaboration and is now among the most developed smart park implementations anywhere. Its "Marvelous City of SIP" app covers the full one-stop service scope in both English and Chinese [56].

Services span business registration, permits, utilities, and cultural amenities. Key transferable principles include a single service access point, mobile-first design, integrated payment workflows, and transparent request tracking — alongside a dependency on reliable connectivity and Chinese payment ecosystems that does not transfer directly.

2.3 Belt and Road Initiative and Digital Silk Road

2.3.1 China-Africa Technology Transfer Context

The Belt and Road Initiative launched in 2013, creating broad frameworks for infrastructure and technology exchange between China and partner countries. The Digital Silk Road — announced in 2015 — extended that specifically into digital infrastructure, e-commerce, and smart city technology [21].

Kluiver and Gagliardone [40] review BRI and DSR implications for African digital development. The picture is mixed: there are genuine opportunities — access to Chinese technology, financing, and working governance models — but also legitimate concerns around technological dependency and contextual fit. Ethiopian industrial parks are deep inside these China-Africa relationships. Chinese capital and expertise built several of them, Chinese companies remain among the largest tenants, and technical ties between Ethiopian and Chinese park managers are ongoing [12, 50]. That relationship creates a practical basis — and a reasonable motivation — for asking whether Chinese digital governance tools could be adapted to work there.

2.3.2 Technology Adaptation in South-South Transfer

The South-South technology transfer literature is consistent on one point: copying does not work. Chen and Liu [16] argue that receiving countries need "absorptive capacity" — the ability to recognise, process, and genuinely apply external knowledge rather than simply import it. Hensengerth [34], studying China-Africa infrastructure transfers specifically, found that projects with localised training, adjusted designs, and continued support tended to succeed where wholesale imports did not. Digital transfers add another layer of difficulty: local digital ecosystems, payment infrastructure, and connectivity conditions all differ, and those differences are not cosmetic.

2.4 Digital Governance in Developing Countries

2.4.1 E-Government Evolution and Models

E-government scholarship has gone through several rounds of rethinking. Layne and Lee [41]'s four-stage model — cataloguing, transactions, vertical integration, horizontal integration — shaped how the field thought about digitisation for a long time. Wimmer [71] pushed focus toward cross-boundary integration in Europe, and Bannister and Connolly [8] traces a more recent shift toward citizen-centred approaches and governance goals that go beyond cost-cutting.

The UN's 2022 E-Government Survey puts Ethiopia in the middle tier on its Development Index. There's real progress on online services, but persistent gaps in telecommunications infrastructure and digital skills [67]. That's roughly where this research sits — not a blank slate, but not ready for off-the-shelf digital governance tools either.

2.4.2 E-Government in Ethiopia

Ethiopia's e-government effort has expanded since the Ministry of Innovation and Technology was established — unevenly, but noticeably. Lessa, Negash and Hailemariam [42] found infrastructure constraints as the biggest barrier, with mobile-based delivery as a partial fix. Denbu and Wondimu [18], looking at public organisations specifically, named connectivity problems, limited digital literacy, and institutional resistance as the dominant obstacles, and came down in favour of mobile-first, offline-capable design as the practical way forward. That's the same design logic this research

follows.

Ethiopia's Digital Strategy 2025 designates industrial parks as priority zones for digital infrastructure investment. Both the national policy and this research are pointing at the same gap.

2.4.3 One-Stop Service Delivery Models

One-stop service models have been implemented widely in e-government. Askim et al. [4] document consistent benefits across national cases: service consolidation, lower administrative burden, better coordination. Bhatnagar [10] found similar efficiency gains and higher satisfaction in South Asian developing country implementations, though inter-agency coordination remained a persistent challenge.

In an industrial park, the one-stop model addresses a very specific problem: tenants currently have to visit different departments for different services. A unified platform removes that burden. The coordination still has to happen, but it happens inside the system rather than falling on the tenant to manage.

2.5 Offline-First Architecture and PWA Technologies

2.5.1 Local-First Software Principles

Kleppmann et al. [39] lay out the local-first software paradigm as a deliberate challenge to conventional web application design. Where standard applications treat the server as the authoritative data source, local-first software makes locally stored data authoritative and uses the network opportunistically for synchronization rather than as a prerequisite for operation.

Seven ideals frame the paradigm [39]: work happens locally without waiting for the network; data is not trapped on a single device; the network is optional; collaboration with colleagues is seamless; data remains accessible over the long term even if service providers disappear; security and privacy are defaults; and users retain ultimate ownership of their data. In environments where connectivity cannot be taken for granted, these principles have direct practical value — people can get their work done regardless of network conditions.

2.5.2 Progressive Web Application Technologies

Progressive Web Applications combine a set of technologies and design patterns that allow web applications to deliver app-like experiences while preserving the reach and accessibility of the web [5, 45]. Tandel and Jamadar [63] document measurable gains in user engagement and retention over traditional web applications, attributing these to three key capabilities: home screen installation, offline operation, and progressive enhancement — functioning on any browser while offering richer features where supported.

Technically, PWAs rest on three pillars. Service workers are JavaScript files that run in a separate thread, enabling background processing, request interception, and caching. Web app manifests supply the metadata needed for installation and customization. Caching APIs — particularly the Cache API and IndexedDB — provide persistent client-side storage for assets and data alike.

Biorn-Hansen, Majchrzak and Gronli [11] compare PWAs with native mobile applications across performance, capability, development cost, and user experience, finding that PWAs deliver comparable results for many application types at significantly lower development and deployment costs. The advantage is especially pronounced when offline capability and multi-platform reach are priorities.

2.5.3 Service Workers and Caching Strategies

Service workers enable the caching strategies that offline-first applications depend on [25]. Google's Workbox library provides production-ready modules for managing service worker logic, simplifying the implementation of common patterns [28]. Archibald [3] catalog these patterns extensively in the "Offline Cookbook," while Ahn and Allen [1] describe the principal approaches: cache-first (serve from cache, fall back to network), network-first (try network, fall back to cache), stale-while-revalidate (serve immediately from cache while updating in the background), and cache-only or network-only strategies for specific resource types.

For offline-first applications, cache-first is generally preferred for assets and previously loaded data, since it guarantees availability regardless of connectivity. Stale-while-revalidate suits data that changes periodically, balancing instant responsiveness

with eventual consistency. Background synchronization, enabled through the Background Sync API, lets PWAs defer network operations until connectivity returns — essential for queueing data submissions created while offline.

2.5.4 IndexedDB and Client-Side Data Persistence

IndexedDB provides a low-level API for storing substantial amounts of structured data on the client. Unlike localStorage — limited to string key-value pairs and typically capped at 5–10 MB — IndexedDB supports complex data structures, indexing, and storage limits that typically reach hundreds of megabytes [48]. In offline-first applications, it serves as the local database: pending changes accumulate there and synchronize with the server when connectivity resumes. Libraries such as Dexie.js [23] abstract over the raw IndexedDB API, simplifying common operations and improving code maintainability. Nicoara [51] offer detailed guidance on offline-first development patterns using these technologies.

2.5.5 Firebase Offline Persistence

Google's Firebase platform provides built-in offline persistence through Firestore. When enabled, Firestore automatically caches data locally and synchronizes changes once the network is available again [27], offloading most synchronization logic to the platform rather than requiring application-level implementation. Conflicts are resolved using a last-write-wins strategy at the document level; more complex resolution can be implemented via Cloud Functions triggered by database changes. Firebase's generous free tier also makes it practical for thesis research and prototype deployment.

2.6 ICT4D and Connectivity Constraints

2.6.1 ICT for Development Frameworks

ICT4D research asks how digital technologies can contribute to development in resource-constrained settings [6, 69]. Heeks [33] describes the field's progression from ICT4D 1.0 — transferring technology from developed countries — through participatory and locally adapted approaches, to a third wave that embeds technology questions within broader development processes [59, 62, 77]. The one finding that holds across

all phases: technology interventions need to fit local conditions. Toyama [66] puts it in terms of amplification — technology magnifies what human and institutional capacity already exists, but it cannot create that capacity from nothing.

2.6.2 Connectivity Challenges in Developing Regions

Connectivity infrastructure in developing regions often diverges fundamentally from the assumptions embedded in technologies designed for reliably connected environments. Chen and Nath [14] document that intermittent access — periods of availability interspersed with outages and variable quality — is the norm, not the exception. Delay-tolerant networking research offers relevant technical frameworks. Fall [24] introduced delay-tolerant networking for scenarios where continuous end-to-end connectivity cannot be assumed; though originally targeting extreme settings like disaster response and interplanetary networks, the underlying principles apply directly to everyday connectivity challenges in developing regions.

2.6.3 Mobile-First and Offline-First in Development Contexts

The rapid spread of mobile devices in developing regions has driven sustained interest in mobile-first design. Donner [19] find that mobile devices frequently serve as people's primary — and sometimes only — computing platform. Offline-first design extends this mobile-first orientation to address connectivity limitations more directly. Muawwal and Khan [49] demonstrate that offline-first application design meaningfully improves both service accessibility and user satisfaction in government service contexts with limited connectivity.

Cherukuri and Patel [17] provide practical implementation guidance for offline-first PWAs, covering synchronization patterns, conflict resolution, and UX design for offline scenarios. A key recommendation: give users clear, continuous feedback about connectivity status and synchronization state.

2.7 Technology Adaptation Frameworks

Two theoretical threads inform the adaptation approach taken here. Appropriate Technology Theory, rooted in Schumacher [61], holds that solutions must suit local conditions — available resources, skill levels, and institutional realities — rather than

assuming that advanced technologies from industrialized settings are universally applicable. Toyama [66] extends this to digital contexts: technology amplifies existing human and institutional capacity; it cannot create what is not already there. Classical technology transfer theory reinforces the point: Rogers [58] identifies absorptive capacity and institutional environment as key determinants of success, while Edgerton [20] shows that adoption is never passive — recipients always adapt technologies as they deploy them.

On the platform side, Schreieck, Wiesche and Krcmar [60] describe adaptation strategies along dimensions of feature modification, integration adaptation, and ecosystem repositioning. The adaptation in this research is substantial on all three: the connectivity architecture changes fundamentally, the payment ecosystem is replaced entirely with an internal token system, and core functionality (service consolidation, workflow automation, status tracking) is preserved while being reshaped for a different institutional context.

2.8 Ethiopian Industrial Parks Context

2.8.1 Industrial Parks Development Corporation (IPDC)

The Industrial Parks Development Corporation was established in 2014 as a public enterprise responsible for developing and managing Ethiopia's industrial parks. It is the institutional instrument behind the country's push to shift its economy from agriculture toward manufacturing [32, 53]. Gebreeyesus [26] traces how this strategy developed, placing the park programme within a wider effort at structural economic transformation.

IPDC's portfolio includes Bole Lemi Industrial Park (the first to become operational, in 2014), Eastern Industrial Zone (developed with Chinese partners), Hawassa Industrial Park (the flagship eco-industrial facility), and several parks at various development stages. These parks host both Ethiopian and international manufacturers, with a notable presence of Chinese, Turkish, and Indian firms [65].

2.8.2 Current Service Delivery Challenges

Guteta and Jiang [31] document recurring shortfalls in administrative responsiveness, infrastructure reliability, and communication at Ethiopian industrial parks. Guteta

and Jiang [30] used SERVQUAL to measure the gap between what tenants expect and what they actually experience, finding shortfalls across every dimension and recommending digital systems as the path toward improvement.

The operational reality is straightforward: processes run on paper. Requesting a service means visiting an office. Approvals follow paper through manual routing. Checking on a request means making a phone call. Tenants who came from places where this is handled digitally — China, India, Turkey — find the contrast difficult to overlook, and research confirms it shapes how they assess IPDC.

2.8.3 Infrastructure and Connectivity Context

Ethiopian telecommunications infrastructure has grown substantially in recent years, with mobile penetration surpassing 50% and internet usage rising quickly. Connectivity remains unreliable outside major cities, however, and industrial parks — despite their strategic importance — are not insulated from regular disruptions. Survey respondents consistently reported three to five hours of daily internet disruption in their parks, with complete outages occurring two to three times monthly. This profile is fundamentally incompatible with cloud-dependent systems that treat network availability as a given.

Government investment in telecommunications — fiber optic expansion, 4G coverage — is underway, but infrastructure improvements take time. Any near-term digital solution must work within current constraints rather than banking on connectivity improvements that have yet to materialize.

2.8.4 China-Ethiopia Industrial Park Relationships

China-Ethiopia ties in industrial park development span investment, construction, tenancy, and technical cooperation [55]. Negesa and Liu [50] document the scale of Chinese involvement in financing and building Ethiopian parks. Chinese companies form a substantial share of tenants — particularly in textiles and garments — meaning park stakeholders already have direct familiarity with Chinese business practices and may be receptive to digital systems informed by Chinese models. Technical cooperation channels between Ethiopian and Chinese park managers provide further routes for knowledge transfer, while also sharpening attention to how much genuine adaptation is

needed to suit Ethiopian conditions.

2.9 Design Science Research Methodology

2.9.1 DSR Paradigm and Principles

Design Science Research is built around the idea that creating and evaluating artifacts is itself a valid form of research [35]. Where behavioural research tries to understand and explain phenomena, DSR tries to solve problems through purposeful design. Seven guidelines govern the approach: the research must produce a working artifact, address a real and relevant problem, evaluate utility rigorously, make clear knowledge contributions, apply methods systematically, treat design as a search through possibilities, and communicate results to appropriate audiences.

2.9.2 DSRM Process and Artifact Types

Peffers et al. [57] propose a six-activity DSRM process model — problem identification, defining objectives, design and development, demonstration, evaluation, and communication — that organizes this research from literature review through thesis. March and Smith [46] identify four artifact types (constructs, models, methods, instantiations); this research produces all three non-trivial types: an instantiation (the platform prototype), a method (the adaptation framework), and constructs (the design principles). Gregor and Hevner [29] argues that design theory — knowing how to do something — is epistemically as valuable as explanatory theory, a position that justifies DSR as the appropriate paradigm where existing solutions fall demonstrably short.

2.10 Related Works and Comparative Analysis

2.10.1 Smart Park Implementations

Several research efforts have examined smart park implementations in various settings. Table 2.1 summarizes the most relevant works and their relationship to this research.

Table 2.1 Related Smart Park Research

Study	Focus	Context	Gap Addressed
Yang et al. (2025)	Park Innovation Mgmt. framework	Chinese smart parks	Conceptual framework only
Huang et al. (2023)	SEZ digitalization	China	Assumes connectivity
Wang et al. (2022)	Smart park platforms	China	Assumes connectivity
Grosse-Bley & Kostka (2021)	Big data in smart cities	Shenzhen, China	Does not address offline
Current Research	**Offline-first adaptation**	**Ethiopia**	**Addresses connectivity constraints**

2.10.2 Offline-First and PWA Applications

Research on offline-first and PWA applications in developing country contexts offers useful technical guidance while revealing gaps this research aims to fill. Table 2.2 summarizes the relevant PWA studies.

Table 2.2 Related PWA and Offline-First Research

Study	Focus	App. Domain	Gap
Cherukuri et al. (2024)	Offline-first PWA patterns	General	Not industrial context
Muawwal et al. (2024)	Offline gov. services	Citizen services	Simple workflows only
Ahn & Allen (2020)	PWA capabilities	General web apps	Not developing country
Current Research	**Offline-first industrial OSS**	**Industrial parks**	**Complex workflows, Ethiopia**

2.10.3 E-Government in Ethiopian Context

Ethiopian e-government research provides important context while making clear the absence of industrial park-specific digital solutions. Lessa, Negash and Hailemariam [42] and Denbu and Wondimu [18] document broader e-government challenges in Ethiopia without addressing industrial park service delivery. This research extends

that body of work into the industrial park domain, tackling the connectivity constraints that hold back conventional e-government approaches.

2.11 Research Gap Analysis

2.11.1 Identified Gaps

Four gaps emerge from the review. Table 2.3 maps them against the literature domains.

Gap 1: Chinese smart park implementations are well documented, but no adaptation framework exists for connectivity-constrained environments. The literature assumes persistent broadband.

Gap 2: Offline-first and PWA research covers simple single-user services. The multi-stakeholder workflows of industrial park service delivery — approvals, status tracking, complaint resolution — have not been addressed.

Gap 3: No purpose-built digital solution exists for Ethiopian industrial parks. E-government research covers Ethiopia but not industrial parks; industrial park research covers Ethiopia but not connectivity constraints.

Gap 4: General offline-first design patterns exist but industrial-specific guidance does not: which features to prioritize for offline use, how to sync multi-stakeholder data, how to present connectivity state to operational users.

Table 2.3 presents a gap matrix mapping how existing literature addresses — or fails to address — the key dimensions of this research.

Table 2.3 Research Gap Matrix

Literature Domain	Smart Park	Offline-First	Ethiopia	Industrial
Chinese Smart Parks	✓	×	×	✓
PWA/Offline-First	×	✓	×	×
Ethiopian E-Gov	×	×	✓	×
ICT4D	×	Partial	Partial	×
Current Research	✓	✓	✓	✓

2.12 Conceptual Framework

Figure 2.1 maps the relationship between the bodies of literature reviewed in this chapter. Chinese smart park models supply the design vocabulary; technology adaptation theory governs how that vocabulary travels; offline-first architecture answers the connectivity constraint; and the Ethiopian IPDC context sets the requirements the resulting platform must satisfy.

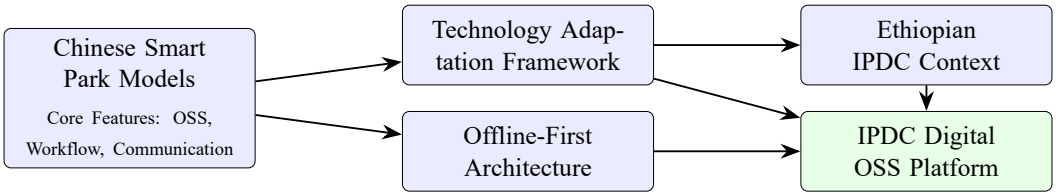

Figure 2.1 Conceptual Framework for Technology Adaptation

Chinese smart park experience defines what digital park governance can look like in practice. The adaptation framework answers what can be transferred and what needs rethinking. Offline-first architecture solves the connectivity problem that makes direct replication impossible. The Ethiopian operating environment sets the requirements that all three must satisfy. The IPDC Digital OSS Platform is where those influences converge.

None of the reviewed literature addresses all three at once: complex multi-stakeholder workflows, industrial park administration, and variable connectivity. That's the gap this research starts from, and the methodology in the next chapter was built specifically around it.

Chapter 3 Research Methodology

3.1 Introduction

Research methodology is worth explaining carefully here, because the choices made were deliberate rather than default. DSR is well established for IT artifact research, but combining it with questionnaire-based requirements gathering and agile-inspired iterative development required thinking through how the pieces fit together. This chapter covers that process: the research design, data collection methods, technology selection rationale, development approach, and evaluation framework.

3.2 Research Paradigm: Design Science Research

3.2.1 Justification of the DSR Approach

DSR suited this problem because the problem doesn't just need explaining — it needs solving. IPDC's paper-based service delivery in a low-connectivity environment is well documented, but documenting it more precisely wasn't the goal. Building something that actually addresses it was. DSR is built for that: designing and evaluating artifacts that tackle real operational problems while producing transferable knowledge alongside them [35].

Three things pushed toward DSR specifically. IPDC needs a working platform, not a theoretical model. No ready-made solution exists for offline-first industrial park governance in an African context, so there was nothing to simply configure or deploy. And the aim wasn't just a working system — it was also generalizable knowledge: design principles, an adaptation framework. DSR is one of the few research paradigms that treats both the artifact and that kind of abstract output as first-class contributions [29].

3.2.2 DSRM Process Model Application

The research follows the six-activity DSRM process model proposed by Peffers et al. [57]. Figure 3.1 shows how these activities map to the research timeline and thesis structure.

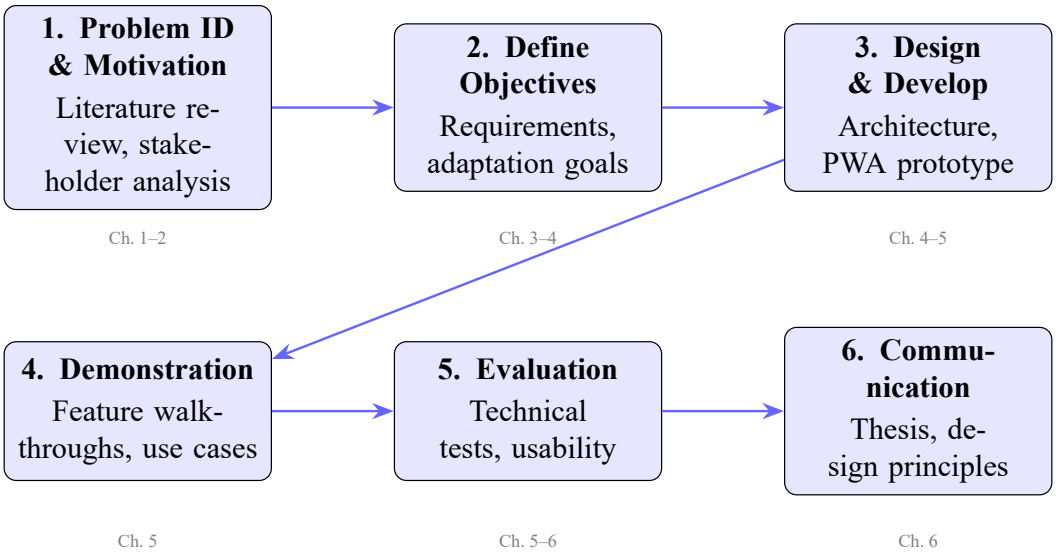

Figure 3.1 DSRM Process Model Applied to This Research

Activity 1 (Problem Identification) drew on the literature review and IPDC service delivery analysis; **Activity 2** (Define Objectives) translated stakeholder requirements and Chinese smart park analysis into design requirements (Chapter 4); **Activity 3** (Design & Development) produced the offline-first PWA prototype; **Activity 4** (Demonstration) showed tenant, administrator, and operations workflows under both online and offline conditions; **Activity 5** (Evaluation) assessed offline functionality, synchronization, performance, and usability via SUS; and **Activity 6** (Communication) is this thesis.

3.3 Research Design Overview

The design combines qualitative and technical methods inside the DSR framework. Stakeholder surveys, document analysis, and Chinese platform review built the picture of what the problem actually was and what a solution needed to do. Iterative devel-

opment and systematic testing produced and validated the artifact itself. DSR doesn't lock you into a specific method — it uses whatever is needed to build and rigorously evaluate the artifact [35].

The work unfolded across four overlapping phases:

1. **Exploration Phase** (Months 1–3): Literature review, Chinese platform analysis, stakeholder requirement gathering, and problem formulation.
2. **Design Phase** (Months 3–5): Architecture design, technology selection, adaptation framework development, and database schema design.
3. **Development Phase** (Months 4–8): Iterative prototype implementation with features added progressively and offline behavior tested continuously.
4. **Evaluation Phase** (Months 8–10): Technical testing, usability assessment, and synthesis of design principles.

3.4 Data Collection Methods

3.4.1 Stakeholder Requirements Questionnaire

Structured questionnaires were administered to representatives of the three main stakeholder groups: tenant firm representatives, IPDC administrative staff, and park operations personnel. The instrument was designed to capture:

- Current service delivery workflows and their pain points;
- How often connectivity disruptions occur and what impact they have;
- Which services stakeholders most want delivered digitally;
- Familiarity with technology and patterns of device use;
- What stakeholders expect from a digital platform.

A total of **49 respondents** completed the questionnaire, exceeding the initial target of 45. Respondents were drawn from **eleven Special Economic Zones and industrial parks** across Ethiopia: Semera, Hawassa, Adama, Bole Lemi, Mekelle, Kilinto, Kombolcha, Bahir Dar, Debre Birhan, and Jimma SEZs, as well as Dire Dawa Free Trade Zone. This geographic spread ensures that the requirements reflect the diversity of park contexts in terms of industry mix, infrastructure maturity, and location. Table 3.1 summarizes the breakdown by stakeholder group.

Table 3.1 Questionnaire Response Summary by Stakeholder Group

Stakeholder Group	Target	Received	Response Rate
Tenant Firm Representatives	20	20	100%
IPDC Administrative Staff	15	15	100%
Park Operations Personnel	10	14	140%
Total	**45**	**49**	**109%**

The questionnaire combined Likert-scale items for quantifiable measurement with open-ended questions for richer qualitative insight. Several findings directly shaped the platform requirements. On service delivery pain points, the overwhelming majority of tenants reported always needing to follow up on submitted requests, with long wait times, lack of status updates, and difficulty reaching the right person as the top three frustrations. On connectivity, most respondents rated internet reliability at their park at 3 out of 5, with outage frequency ranging from "sometimes (1–2 times per week)" for field operations staff to "sometimes (1–4 times per month)" for administrative staff. On digital readiness, all ten features in the platform feature survey received average importance ratings above 4.0 out of 5; offline capability specifically was rated "5 – Very Important" by nearly all respondents across all three groups. Language preference was overwhelmingly "Both English and Amharic," a finding that supports Amharic localization as a priority for future development. On platform adoption intent, only two respondents expressed uncertainty, while the remainder selected "Yes, definitely."

3.4.2 Document Analysis

Existing IPDC documentation was reviewed to understand formal service processes, organizational structures, and regulatory requirements. Documents examined included service request forms, fee schedules, organizational charts, operational guidelines, and annual reports. This analysis fed into the functional requirements and helped ensure that the prototype aligns with established institutional processes.

3.4.3 Platform Analysis: WeChat Work and DingTalk

A systematic analysis of Chinese smart park platforms — primarily WeChat Work and DingTalk — identified features, interaction patterns, and architectural approaches

that could inform the IPDC platform design. The analysis drew on platform documentation, published research, and publicly available demonstrations, with features categorized by their relevance and transferability to the Ethiopian context.

The analysis established the source model for the technology adaptation framework while also generating concrete design ideas — workflow automation, approval routing, status tracking — for the IPDC platform.

3.4.4 Contextual Insights from Open-Ended Responses

Each questionnaire included open-ended questions alongside the Likert-scale items, specifically to capture contextual detail that closed-format items cannot surface. Respondents were invited to describe, in their own words:

- How service requests are currently submitted and tracked in their park;
- The typical impact of internet disruptions on their daily work;
- What they found most frustrating about existing administrative processes;
- What features they would most want a digital platform to provide.

In practice, the open responses were more revealing than the Likert scores. People described using WhatsApp to submit requests, keeping personal notebooks to track what they had submitted and when, and calling contacts directly when the formal process stalled. Those workarounds said something important about what any digital replacement would need to do, and they shaped the design decisions in Chapter 4 in ways a five-point scale could not.

3.5 System Development Methodology

3.5.1 Agile-Inspired Iterative Development

The prototype was developed using an agile-inspired iterative approach. Rather than following a rigid waterfall sequence, development proceeded through multiple iterations, each adding functionality and incorporating lessons from testing. This suits DSR well, since the artifact takes shape through an iterative cycle of design and evaluation [35].

Each iteration followed a build-test-refine cycle:

1. **Build:** Implement a set of features drawn from prioritized requirements.

2. **Test:** Verify correct operation, with particular attention to offline behavior and cross-platform compatibility.
3. **Refine:** Fix issues and adjust the design based on what testing revealed.

Iterations were organized around functional modules: authentication and user management; service request workflows; the digital token system; announcements and complaint management; AI model integration; and the interactive park map.

3.5.2 Technology Selection Rationale

Three questions governed every technology choice: does it support offline-first operation, is it mature enough to rely on, and does it stay within a thesis research budget? Table 3.2 summarizes the technologies selected and the reasoning behind each.

Table 3.2 Technology Stack Summary

Category	Technology	Selection Rationale
Frontend Framework	React 19 + TypeScript	Component-based architecture, strong PWA ecosystem, type safety
Build Tool	Vite 7	Fast dev server, native PWA plugin support via vite-plugin-pwa
UI Library	Material-UI (MUI) 7	Comprehensive component library, responsive design, accessibility
Backend	Firebase 12 (Firestore, Auth)	Built-in offline persistence, generous free tier, real-time sync
Local Database	Dexie.js 4 (IndexedDB)	High-level IndexedDB wrapper, React hooks integration
PWA Tooling	Workbox 7 + vite-plugin-pwa	Industry-standard service worker management and caching
AI Backend	FastAPI (Python)	Lightweight REST API, Pydantic validation, easy model deployment
Maps	Leaflet + React-Leaflet	Open-source mapping, no API key required, offline tile support
Forms	React Hook Form + Zod	Performant form handling, schema-based validation
Charts	Recharts	React-native charting, responsive, minimal dependencies
PDF Generation	jsPDF + AutoTable	Client-side PDF generation for invoices and reports

React and TypeScript provide a component model suited to multi-stakeholder complexity, with compile-time type safety across eight type definition files. **Firebase**

was chosen over Supabase or a custom backend specifically for `persistentLocalCache()`, which automatically handles offline data and bidirectional synchronization without custom sync logic. **Dexie.js** adds an explicit write queue for operations that Firestore's optimistic cache cannot guarantee during genuine offline periods; together they form the dual-layer storage architecture. **Workbox** handles service worker management through declarative CacheFirst, NetworkFirst, and StaleWhileRevalidate configurations, avoiding the subtle errors that manual service worker code reliably introduces.

3.5.3 Development Tools and Environment

Development used Visual Studio Code as the primary IDE, with Vite's development server providing hot module replacement for fast iteration. Firebase emulators handled local testing of authentication and Firestore operations without consuming cloud resources. The AI model backend was built in Python with FastAPI, and models were trained using scikit-learn and XGBoost on structured datasets representing IPDC operational scenarios.

Version control was managed through Git. The production build pipeline uses Vite with manual chunk splitting to keep bundle sizes manageable — vendor libraries (React, ReactDOM), the UI framework (Material-UI), and Firebase SDKs are separated into distinct chunks for better caching.

3.6 Evaluation Framework

An artifact that works correctly but frustrates users, or one that users like but whose offline architecture fails, would both fall short of what DSR demands [35]. The evaluation therefore covers both technical operation and user experience, following the FEDS framework proposed by Venable, Pries-Heje and Baskerville [68], which pairs formative and summative strategies.

3.6.1 Technical Evaluation

The central technical question is simple: does the offline-first architecture actually work under conditions that resemble real use? Technical evaluation checks the prototype against its design specifications across four dimensions:

1. **Offline Functionality:** Each core feature is systematically tested under simulated offline conditions to confirm that users can complete critical workflows without connectivity.
2. **Synchronization Accuracy:** Data created or modified offline is verified to synchronize correctly with Firestore once connectivity returns, with no data loss or corruption.
3. **Performance:** Key metrics are measured, including initial load time, cached load time, response time during offline operation, and synchronization duration.
4. **Cross-Platform Compatibility:** The prototype is tested on desktop browsers (Chrome, Firefox), mobile browsers (Chrome for Android, Safari for iOS), and as an installed PWA.

3.6.2 Usability Evaluation

Usability is assessed using the System Usability Scale (SUS), a widely validated instrument for measuring perceived system usability [13, 52]. The instrument distils ten alternating positive/negative statements into a single 0–100 score; by convention, anything above 68 sits in above-average territory. Lewis [43] confirm the instrument's reliability across three decades of application, making it well suited for evaluating novel systems with no established baseline.

Representative stakeholders from each user group (tenants, administrators, operations personnel) worked through structured task scenarios on the prototype before completing the SUS questionnaire. Qualitative feedback was also gathered through follow-up questions about specific interface aspects and the offline experience.

3.6.3 DSR Guidelines Compliance

As a methodological quality check, the research evaluates its own compliance with Hevner's seven DSR guidelines. Table 3.3 maps evaluation criteria to the guidelines and specifies the evidence required for each.

Table 3.3 Evaluation Criteria Matrix

DSR Guideline	Evaluation Criterion	Evidence
1. Design as Artifact	Functional prototype exists	Working PWA with documented features
2. Problem Relevance	Addresses real IPDC needs	Requirements traced to stakeholder input
3. Design Evaluation	Rigorous assessment	Technical tests + SUS usability scores
4. Research Contributions	Novel knowledge produced	Adaptation framework + design principles
5. Research Rigor	Systematic methods applied	DSRM process followed, documented
6. Design as Search	Iterative refinement	Multiple development iterations
7. Communication	Results disseminated	Thesis document, chapter structure

3.7 Ethical Considerations

The research adheres to ethical principles throughout. Participation in requirements gathering and usability evaluation was voluntary, and participants were informed of the research purpose and their right to withdraw at any point. No personally identifiable data from real IPDC tenants or staff was used in the prototype; all demonstration data is synthetic. The authentication system stores credentials securely through Firebase Authentication, and offline credential caching uses hashing rather than plaintext storage.

The platform is also designed with organizational ethics in mind. Administrative staff remain at the center of service delivery — the platform gives them better tools rather than automating them out of the picture.

With this methodological grounding in place, the next chapter moves directly into what the data revealed and what the design produced.

Chapter 4 System Analysis and Design

4.1 Introduction

Getting from stakeholder pain points to a working system design is rarely a linear process. This chapter records what that process produced for the IPDC platform: the requirements that emerged from stakeholder analysis, the framework for adapting Chinese smart park patterns to the Ethiopian context, and the architecture that resulted. Not every decision was obvious at the time — but all of them trace back to a documented constraint or a real operational need.

4.2 Requirements Analysis

4.2.1 Stakeholder Identification and Analysis

Reading IPDC's organizational structure alongside the questionnaire data revealed four distinct stakeholder groups, each with a different relationship to the platform and different expectations from it.

Tenant Enterprises submit service requests, track progress, and handle fee payments; speed, transparency, and offline operation are their chief concerns.

IPDC Administrative Staff process requests, publish announcements, manage tenant accounts, and resolve complaints; workflow efficiency and reliable record-keeping matter most.

Operations Personnel receive and complete task assignments in the field — often in areas with unreliable connectivity — making the offline task queue directly relevant to their working day.

IPDC Management needs aggregated operational data (resolution times, park performance, resource utilization) rather than direct involvement in individual workflows.

4.2.2 Functional Requirements

Table 4.1 summarizes the twelve functional requirement groups that emerged from stakeholder analysis and the mapping of Chinese smart park features.

Table 4.1 Functional Requirements Summary

ID	Requirement	Description
FR1	User Authentication	Email-based registration and login with role assignment (tenant, admin, operations); offline authentication fallback using cached credentials
FR2	Service Request Management	Submit, track, approve, and complete service requests across multiple categories with file attachments and priority levels
FR3	Digital Service Token System	Internal credit system for fee tracking with tiered accounts (basic, silver, gold, platinum), transaction history, and automated deductions
FR4	Announcement Management	Create, distribute, and display system announcements targeted to specific audiences with priority and expiration settings
FR5	Complaint Management	File complaints against completed services with evidence uploads, track resolution through defined workflows
FR6	Asset & Maintenance Management	Register, categorize, and track park assets; schedule and record maintenance activities
FR7	AI-Powered Services	Automated service request classification, predictive equipment maintenance, and intelligent park recommendation for prospective tenants
FR8	Document Management	Upload, store, and retrieve documents associated with service requests and OSS applications
FR9	OSS Services	Digital interfaces for investment permits, business licenses, customs services, and banking coordination
FR10	Interactive Park Map	Geographic visualization of Ethiopian industrial parks with detail overlays
FR11	Billing & Invoicing	Generate invoices, track payment transactions, and export PDF reports
FR12	Notifications	In-platform and email notifications for status changes and announcements

4.2.3 Non-Functional Requirements

Table 4.2 sets out the non-functional requirements that constrain and shape the system architecture.

Table 4.2 Non-Functional Requirements

ID	Requirement	Specification
NFR1	Offline-First Operation	All core workflows (FR1–FR5) must function without connectivity; data persists locally and synchronizes when online
NFR2	Cross-Platform Access	Single codebase accessible on desktop, tablet, and mobile via web browser; installable as PWA on mobile devices
NFR3	Response Time	Page transitions under 2 seconds; form submissions under 1 second in offline mode
NFR4	Sync Latency	Queued data synchronized within 30 seconds of connectivity restoration
NFR5	Security	Role-based access control; Firebase Authentication; credential hashing for offline cache
NFR6	Scalability	Architecture supports multiple industrial parks under a single deployment
NFR7	Accessibility	WCAG 2.1 Level AA compliance for core interfaces

4.2.4 MoSCoW Prioritization

The MoSCoW framework guided requirement prioritization and iteration planning. Offline availability served as the primary deciding factor. Table 4.3 shows the results.

Table 4.3 MoSCoW Feature Prioritization

Priority	Features	Offline Requirement
Must Have	Authentication, service requests, token system, announcements, complaint management	Full offline capability
Should Have	Asset management, notifications, billing, document management	Graceful degradation
Could Have	AI services, interactive map, OSS services, PDF export	Online-preferred
Won't Have (this phase)	External payment gateway, IoT integration, Amharic localization	Identified as future work

4.3 China-to-Ethiopia Technology Adaptation Framework

4.3.1 Chinese Smart Park Feature Analysis

WeChat Work and DingTalk share a recognizable feature set: enterprise messaging, workflow approval routing, service request submission and tracking, payment processing (WeChat Pay or Alipay), document management, attendance and access control, video conferencing, and analytics dashboards.

4.3.2 Adaptation Dimensions

Four dimensions structure the adaptation framework. The technological dimension carries the most weight — the connectivity gap between Chinese and Ethiopian contexts is the primary constraint that reshapes everything else — but organizational fit, cultural context, and economic realities each produce their own distinct adaptation requirements.

Technological Dimension: Connectivity demands the most fundamental adaptation. Chinese platforms assume persistent high-speed internet; the Ethiopian version replaces this with an offline-first architecture. Dependencies on Chinese ecosystem platforms (WeChat, Alipay, Alibaba Cloud) are replaced by globally accessible alternatives (Firebase, web standards).

Organizational Dimension: IPDC's organizational structure and service processes do not mirror those of Chinese park management. The adaptation translates Chinese workflow patterns into IPDC's three-role structure (tenant, admin, operations) and aligns approval workflows with the institution's existing hierarchies.

Cultural Dimension: Interface design was adjusted to fit local expectations. An English-language interface reflects the international business environment of Ethiopian industrial parks, while Amharic localization has been identified as a future enhancement. The digital service token concept is an adaptation of Chinese integrated payment, designed for a context where external payment gateway partnerships have not yet been established.

Economic Dimension: Cost constraints shaped technology choices at every stage: PWA over native apps (eliminating app store fees and multi-platform development costs), Firebase's free tier over paid cloud services, and open-source tools across the full stack.

4.3.3 Feature Mapping and Modification Matrix

Table 4.4 maps each Chinese smart park feature to its adapted counterpart in the IPDC platform.

Table 4.4 China-to-Ethiopia Feature Adaptation Matrix

Chinese Feature	IPDC Adaptation	Modification Rationale
WeChat Work messaging	Announcement system	Structured notifications replace real-time chat; works offline
DingTalk approval workflows	Service request pipeline	Adapted to tenant→admin→operations flow
WeChat Pay / Alipay	Digital service tokens	Internal system avoids external API dependency
Native mobile apps	Progressive Web App	Single codebase, no app store, offline-capable
Cloud-only architecture	Offline-first with Firebase	Dual-layer local storage + cloud sync
Real-time IoT monitoring	Asset management module	Manual tracking as IoT infrastructure unavailable
Chinese language UI	English interface	Primary business language; Amharic as future work
Alibaba Cloud backend	Firebase (Google Cloud)	Global availability, built-in offline persistence

4.3.4 Offline-First Adaptation Strategy

The offline-first adaptation goes deeper than anything else in the framework. It is not a feature to switch on — it changes how data flows through the whole system, from the moment a user takes an action to the moment that action reaches the server. The adaptation is built on three guiding principles:

1. **Local-first data storage:** Every piece of data the user reads comes from local storage (the Firestore cache or IndexedDB), while network requests happen in the background to keep things synchronized.
2. **Queue-based write operations:** When users create or modify data while offline, their operations are placed in an IndexedDB queue and executed against Firestore once connectivity is restored.
3. **Transparent status communication:** The interface continuously shows con-

nectivity status, synchronization state, and any pending operations, so users always know exactly where their data stands.

4.4 System Architecture

4.4.1 High-Level Architecture Overview

The platform is organized as a three-tier architecture adapted for offline-first operation. Figure 4.1 provides an overview of the high-level structure.

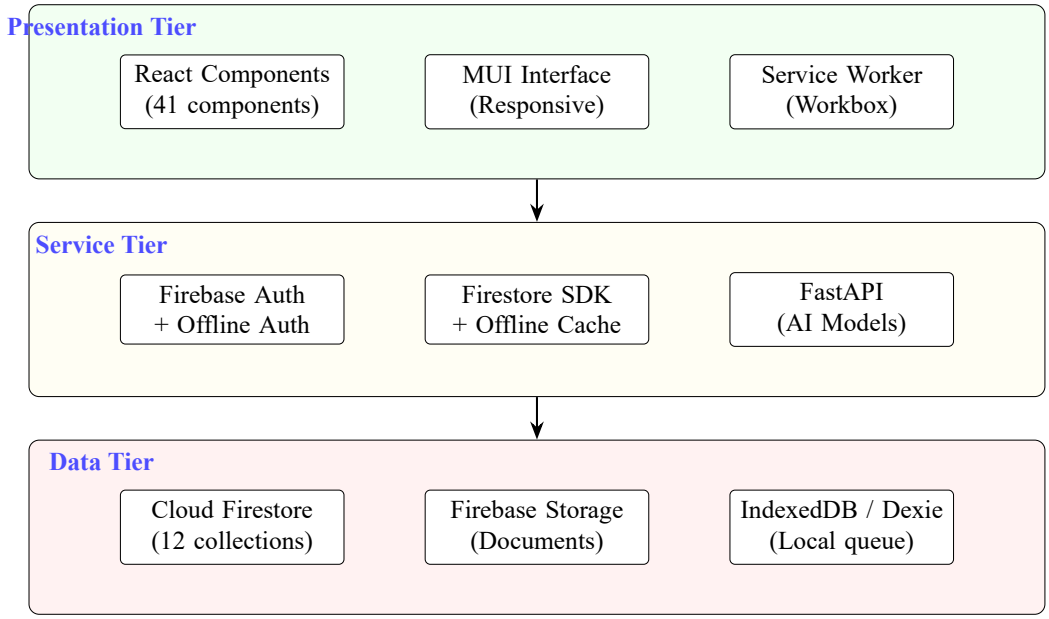

Figure 4.1 High-Level System Architecture

The **Presentation Tier** comprises 41 React components (six modules + 15 pages), with Material-UI for responsive design and Workbox for service worker management. The **Service Tier** connects the UI to storage: Firebase Auth handles online verification with an offline fallback against cached IndexedDB credentials, the Firestore SDK provides transparent caching, a custom sync service manages the write queue, and FastAPI serves the three AI models. The **Data Tier** blends cloud Firestore (12 collections), Firebase Storage (documents and images), and IndexedDB via Dexie.js (local queue and cached data).

4.4.2 Offline-First Architecture Design

The offline-first architecture relies on a dual-layer storage strategy. Figure 4.2 shows data flow during a service request submission under both online and offline conditions.

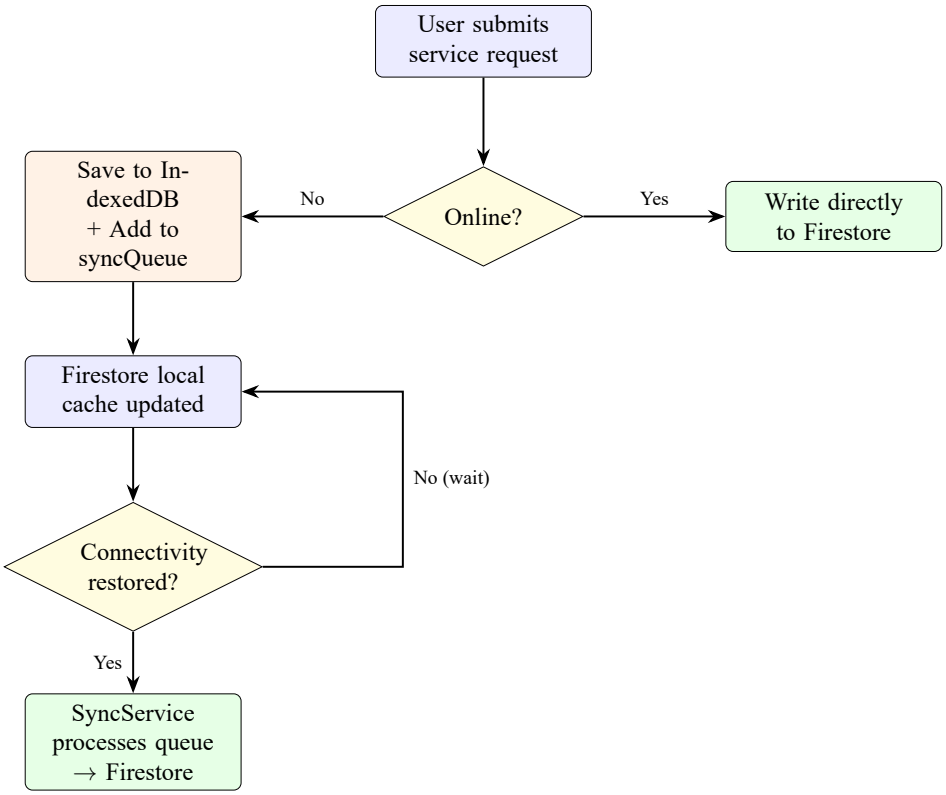

Figure 4.2 Offline-First Data Flow for Service Request Submission

Layer 1 — Firebase Firestore Persistent Cache: With `persistentLocalCache()` and `persistentMultipleTabManager()` enabled, Firestore maintains a local copy of every document the user has previously accessed. When offline, read operations draw from this cache, giving users seamless access to existing records without any network connection.

Layer 2 — Dexie.js IndexedDB Queue: Write operations initiated while offline are stored in two IndexedDB tables managed by Dexie.js. The `serviceRequests` table holds complete request data locally; the `syncQueue` table records metadata for each pending operation (type, collection, document ID, timestamp). Once the `useOnlineS-`

`tatus` hook detects restored connectivity, the sync service processes queued items and writes them to Firestore.

4.4.3 PWA Architecture

The PWA setup, managed through vite-plugin-pwa, employs three caching strategies:

- **Precaching:** JavaScript bundles, CSS stylesheets, HTML pages, icons, and image assets are cached when the service worker is first installed, ensuring the application shell loads even without network access.
- **CacheFirst** (Google Fonts): Font resources are served from cache with a one-year expiration, avoiding unnecessary repeat downloads.
- **NetworkFirst** (Firebase Storage, API endpoints): For dynamic content, the application tries the network first and falls back to cached versions during outages. Firebase Storage responses are cached for seven days.

The web app manifest declares the application as `"display": "standalone"`, enabling full-screen installation on mobile devices. It also specifies the application name (“IPDC-OSS Platform”), a theme color (`#2563eb`), and icon assets at 192px and 512px resolutions.

4.4.4 Authentication Architecture

Authentication follows a dual-mode design so that users can still log in during connectivity disruptions. Figure 4.3 illustrates the authentication flow.

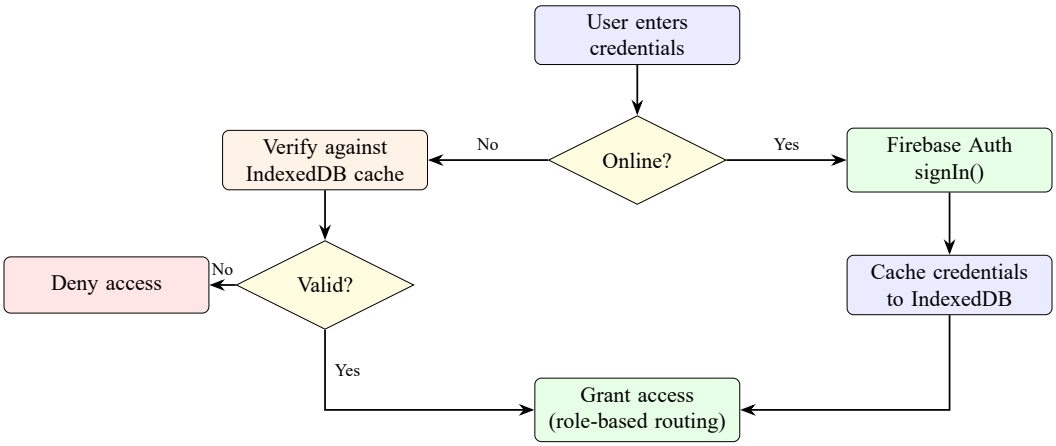

Figure 4.3 Dual-Mode Authentication Flow

When online, authentication proceeds through Firebase Auth via `signInWithEmailAndPassword()`. After a successful login, credentials are hashed and stored in IndexedDB through the `cacheCredentials()` utility. When offline, the system retrieves cached credentials and checks the user's input against the stored hash. The `AuthContext` component orchestrates this dual-mode logic, switching automatically to offline authentication whenever it encounters network errors.

Three user roles — `tenant`, `admin`, and `operations` — determine which dashboard a user sees and which features they can access, managed through role-based conditional rendering in the React component tree.

4.4.5 AI Integration Architecture

Three machine learning models are served through a FastAPI backend as REST endpoints at `/api/v1/`:

1. **Service Classifier** (`/classify-service`): Accepts a request title and description; returns a predicted service type, priority level, estimated processing time in days, confidence score, and contextual recommendations.
2. **Predictive Maintenance** (`/predict-maintenance`): Takes asset attributes — age, condition score, maintenance history — and outputs a failure probability, risk level, estimated days until failure, and a recommended action. The output is designed to give operations teams something actionable rather than just a probability number.

3. **Park Recommendation** (`/recommend-parks`): Accepts a tenant profile (industry sector, employee count, investment capital, infrastructure needs); returns ranked park recommendations with match scores, pros and cons, and cost estimates. Scoring weights are: industry alignment 30%, capacity 25%, infrastructure 20%, cost 15%, location 10%.

AI features are designed with graceful degradation: when the FastAPI backend is unreachable during connectivity outages, the platform reverts to manual classification and disables recommendation features without blocking core workflows.

4.5 Database Design

4.5.1 Firestore Collections Schema

The Firestore database is organized into twelve collections. Table 4.5 documents the primary collections with their key fields and purposes.

Table 4.5 Firestore Collections Schema

Collection	Key Fields	Purpose
users	uid, email, displayName, role, parkId, createdAt	User profiles and roles
serviceRequests	id, tenantId, title, description, serviceType, priority, status, assignedTo	Core service workflow
ossServiceRequests	id, tenantId, serviceCategory, documents[], status	OSS-specific requests
announcements	id, title, content, targetAudience, priority, expiresAt, isActive	System announcements
complaints	id, requestId, tenantId, category, evidence[], status, resolution	Complaint tracking
tokenAccounts	tenantId, balance, tier, totalEarned, totalSpent	Token balances
tokenTransactions	id, tenantId, type, amount, description, timestamp	Transaction ledger
invoices	id, tenantId, lineItems[], total, status, pdfUrl	Billing records
paymentTransactions	id, tenantId, amount, method, referenceNumber	Payment tracking
assets	id, name, category, condition, purchaseCost, currentValue	Park asset inventory
maintenanceRecords	id, assetId, type, scheduledDate, completedDate, cost	Maintenance history
notifications	id, userId, title, message, read, createdAt	User notifications

4.5.2 IndexedDB Schema (Dexie.js)

The local IndexedDB database, named `IPDCDatabase`, holds two tables tailored for offline operations:

- `serviceRequests`: Stores complete request documents for offline access. Indexed on: `id`, `tenantId`, `status`, `priority`, `serviceType`, `createdAt`.
- `syncQueue`: Stores pending write operations with auto-incrementing IDs. Indexed on: `++id`, `type`, `collection`, `status`, `createdAt`.

The sync queue recognizes four status values: `pending` (awaiting sync), `syncing` (currently being processed), `synced` (successfully written to Firestore), and `failed` (sync attempted but unsuccessful; will be retried).

4.6 User Interface Design

The interface follows Material Design through MUI, with a professional blue (`#2563eb`) primary and confident green (`#059669`) secondary, and status-specific colors (amber pending, blue approved, purple in-progress, green completed, red rejected). Responsive breakpoints at 600px and 960px cover all device categories via the `useDeviceDetection` hook.

After authentication, each user is routed to a role-appropriate dashboard: **Tenant** (submit and track requests, manage tokens, file complaints, view park recommendations); **Admin** (manage all requests and announcements, handle tenant accounts and complaints, oversee billing and assets); **Operations** (fulfill tasks, schedule maintenance, inspect assets).

The persistent `StatusBar` shows connectivity status, device type, pending sync count, and last sync timestamp — essential feedback for an offline-first system where users need to know whether actions are queued locally or already written to the server.

How those design decisions translate into working software — and what real stakeholders made of it — is the subject of Chapter 5.

Chapter 5 Implementation and Evaluation

5.1 Introduction

This chapter is, in many ways, where the design decisions of the previous chapter either hold up or fall apart. What was built is a working Progressive Web Application comprising roughly 105 TypeScript source files, three trained machine learning models, and a dual-layer offline data management architecture. Whether it actually works as intended is the question the evaluation half of this chapter answers — through offline functionality tests, performance measurement, cross-platform compatibility checks, and a structured usability assessment with nine real IPDC stakeholders.

5.2 Development Environment and Tools

5.2.1 Development Setup

The development environment centered on Visual Studio Code as the primary IDE, Node.js (v18+) as the JavaScript runtime, React 19.2.1 [47] as the frontend framework, and the Vite development server for hot module replacement at `localhost:5173`. Firebase emulators handled local testing of authentication and Firestore operations, avoiding unnecessary consumption of cloud quotas. The AI model backend was developed separately in a Python environment using FastAPI, with models available at `localhost:8000`.

5.2.2 Project Structure

The project follows a modular directory structure organized by concern. Application source code lives in `src/`, while machine learning components and the API server reside in `ai-models/`. Table 5.1 provides an overview of the directory layout and component counts.

Table 5.1 Project Directory Structure and Component Counts

Directory	Purpose	Files
src/components/admin/	Admin-specific UI components	6
src/components/common/	Shared UI components (StatusBar, FileUpload, etc.)	12
src/components/layout/	Navigation and layout scaffolding	3
src/components/map/	Interactive park map and details	3
src/components/operations/	Operations module components	4
src/components/tenant/	Tenant-specific components	13
src/pages/	Page-level route components	15
src/services/	Business logic and API services	18
src/services/offline/	Offline storage and sync services	3
src/contexts/	React context providers	1
src/hooks/	Custom React hooks	4
src/db/	Dexie.js schema definition	1
src/types/	TypeScript type definitions	8
src/config/	Firebase and theme configuration	2
ai-models/	Three ML models + FastAPI backend	12
Total		**∼105**

5.2.3 Build and Deployment Configuration

The Vite build uses manual chunk splitting (`vendor`, `mui`, `firebase`) so application updates do not invalidate cached library bundles — valuable where connectivity is limited.

The vite-plugin-pwa generates the service worker and web app manifest automatically from the configuration in `vite.config.ts`. The deployed platform is accessible at the following URLs:

- **Frontend Application:** `https://ipdc-oss-platform.vercel.app/` — deployed on Vercel with HTTPS, global CDN distribution, and automatic SSL certificate management.
- **AI Backend API:** `https://ipdc-oss-platform.onrender.com/api/docs` — deployed on Render, hosting the FastAPI server with the three trained ML models. Interactive API documentation (Swagger UI) is available at this URL; model endpoints are served under the `/api/v1/` prefix (e.g., `/api/v1/classify-service`, `/api/v1/predict-maintenance`, `/api/v1/recommend-parks`).

5.3 Key Feature Implementation

5.3.1 Offline-First Architecture Implementation

The offline-first architecture is realized through three cooperating subsystems: Firebase Firestore's transparent cache, the Dexie.js local database, and a custom synchronization service.

Firebase Offline Persistence is activated in the Firebase configuration module (`src/config/firebase.ts`) during initialization:

Listing 5.1 Firebase Offline Persistence Configuration

```
initializeFirestore(app, {
  localCache: persistentLocalCache({
    tabManager: persistentMultipleTabManager()
  })
});
```

This instructs Firestore to maintain a persistent local cache of all documents the user has accessed, keeping it intact across browser sessions and coordinating access when multiple tabs are open. When the network is unavailable, reads are served from this cache and writes are internally queued for later synchronization.

Dexie.js Local Database extends Firestore's cache with explicit offline data management. The schema, defined in `src/db/schema.ts`, sets up two indexed tables:

Listing 5.2 Dexie.js Database Schema

```
const db = new Dexie('IPDCDatabase');
db.version(1).stores({
  serviceRequests:
    'id, tenantId, status, priority, serviceType, createdAt',
  syncQueue:
    '++id, type, collection, status, createdAt'
});
```

The Sync Service (`src/services/offline/syncService.ts`) picks up queued items once connectivity is detected. It iterates through pending entries, converts local JavaScript `Date` objects to Firestore `Timestamp` objects, writes to the appropriate collection, and marks each item with an updated status. A built-in retry mechanism handles transient failures so no queued data is lost.

5.3.2 Authentication with Offline Fallback

The authentication system implements the dual-mode design outlined in Chapter 4. The `AuthContext` (`src/contexts/AuthContext.tsx`) wraps the entire application and manages authentication state. When signing in online, credentials are verified through Firebase Auth and cached locally:

Listing 5.3 Credential Caching After Online Authentication

```
// After successful Firebase Auth sign-in:
await cacheCredentials(email, password);
await cacheAuthState({
  uid: user.uid,
  email: user.email,
  displayName: user.displayName,
  role: userData.role
});
```

If the sign-in attempt fails because of a network error, the context automatically falls back to offline verification:

Listing 5.4 Offline Authentication Fallback

```
// Network error detected during sign-in:
const isValid = await verifyOfflineCredentials(email, password)
    ;
if (isValid) {
  const cachedUser = await getCachedAuthState();
  // Grant access using cached user profile
}
```

Any user who has previously authenticated while online can therefore continue using the platform during connectivity outages. Figure 5.1 shows the sign-in and tenant registration interfaces.

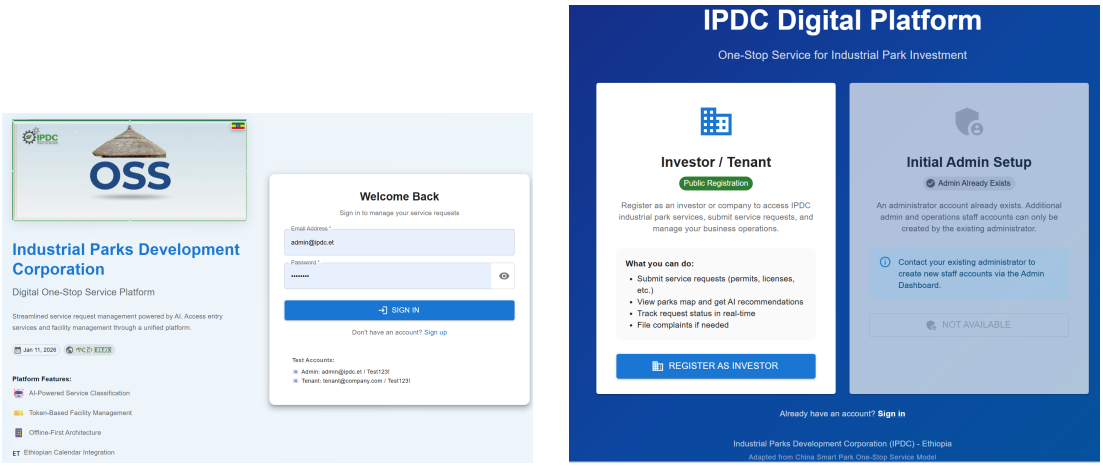

Figure 5.1 Authentication Interfaces — Sign-In (left) and Tenant Registration (right)

5.3.3 Service Request Management

Service request management forms the backbone of the platform. The `CreateRequestForm` component (`src/components/tenant/`) uses React Hook Form for state management and Zod for schema validation. Requests can be categorized across multiple service types (maintenance, IT support, security, cleaning, waste management, and others), assigned one of four priority levels, given free-text descriptions, and accompanied by file attachments uploaded through the `FileUpload` component to Firebase Storage.

Each request follows a defined status progression: `pending` → `approved` → `in-progress` → `completed`. Administrators may also `reject` a request with a stated reason. Every status transition triggers a notification and updates the record in Firestore — or in the local queue when offline. Figure 5.2 shows the tenant dashboard alongside the service request creation form.

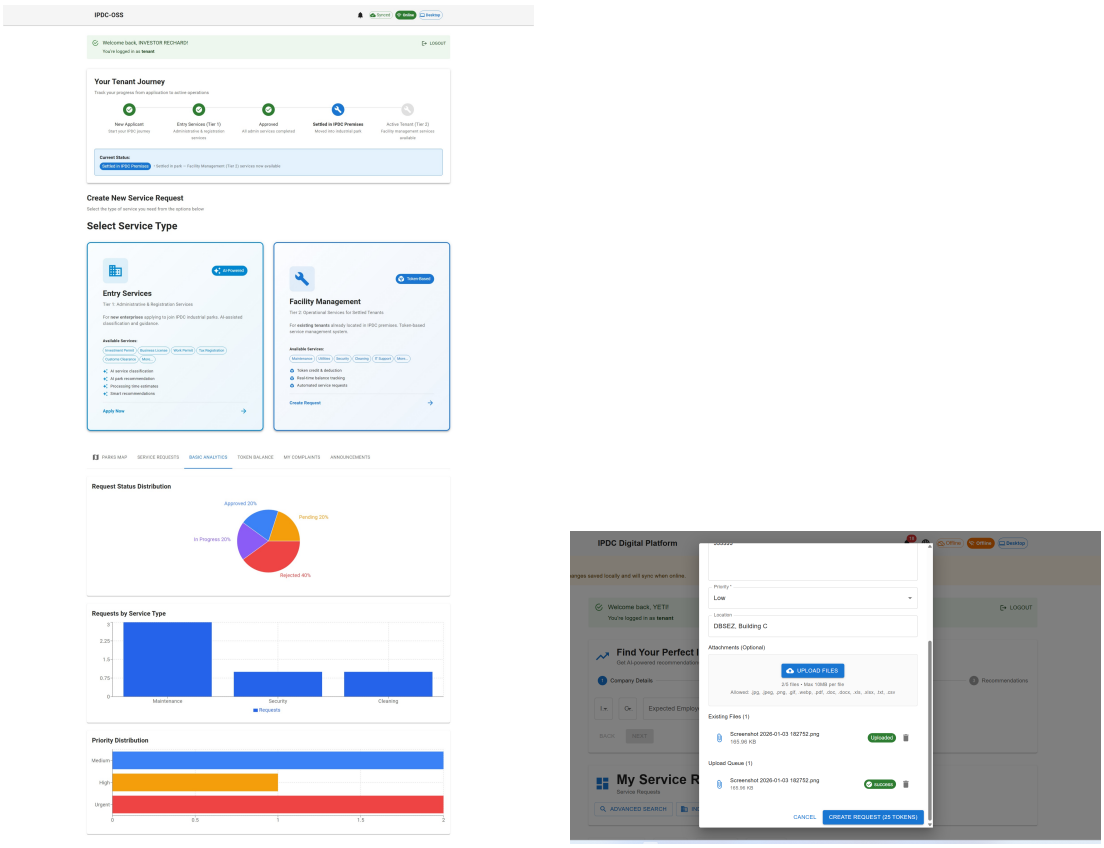

Figure 5.2 Tenant Service Request Workflow — Dashboard Overview (left) and Request Creation Form (right)

5.3.4 Digital Service Token System

The token system provides an internal mechanism for tracking service fees without requiring an external payment gateway. Its implementation spans the `TokenDashboard` component and the `billingService` module.

Token Tiers reward engagement: basic through platinum (four tiers), each unlocking progressively larger discounts (0–20%), higher monthly request limits, and priority support.

Service Costs vary by both service type and priority level. Table 5.2 shows the pricing matrix.

Table 5.2 Service Token Pricing Matrix (tokens per request)

Service Type	Low	Medium	High	Urgent
Maintenance	50	80	120	150
IT Support	60	100	140	180
Security	40	65	95	120
Cleaning	25	45	65	80
Waste Management	35	60	85	110

A complete audit trail covers all transaction types (allocation, deduction, refund, bonus, penalty, purchase). Administrators credit tokens after confirming external payment receipt. Figure 5.3 shows the token dashboard interface.

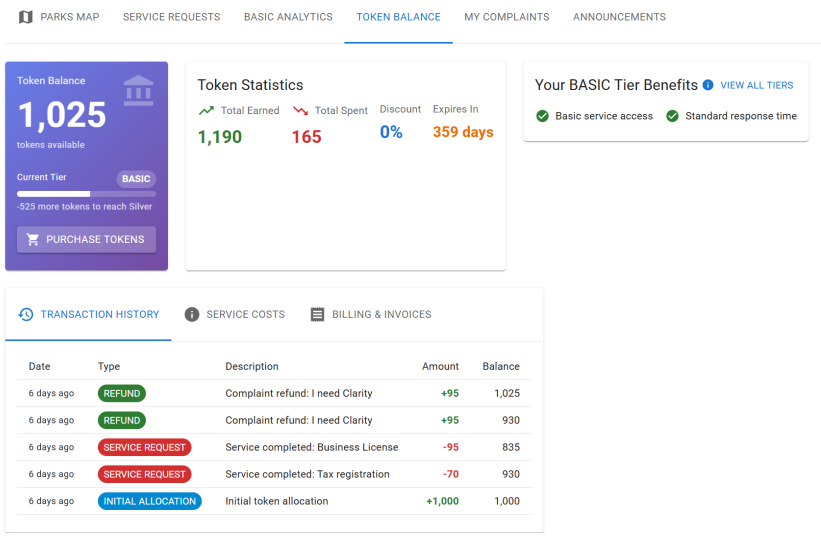

Figure 5.3 Digital Service Token Dashboard

5.3.5 AI Model Integration

Three machine learning models are connected to the platform through a FastAPI backend. The frontend AI service module (`src/services/aiService.ts`) exposes typed interfaces for each model endpoint, with error handling and timeout management built in.

Model 1 — Service Classifier: Trained on labeled service request data, this model sorts incoming requests into eleven service categories and assigns priority levels. Because the endpoint returns a confidence score, the interface presents predictions as sug-

gestions that users can accept or override.

Model 2 — Predictive Maintenance: This was the model stakeholders responded to most immediately during the usability evaluation — the idea of warning operations staff before equipment fails, rather than after, resonated without any prompting. Given asset attributes, it produces a failure probability, risk level (low/medium/high), estimated days until failure, and a recommended maintenance action.

Model 3 — Park Recommendation: Built to help prospective tenants choose a suitable industrial park, this model evaluates a tenant profile against park characteristics using a weighted scoring algorithm. Five dimensions — industry alignment (30%), capacity and availability (25%), infrastructure match (20%), cost efficiency (15%), and location preference (10%) — produce ranked recommendations with detailed justifications.

All three models degrade gracefully: if the API is unreachable, the interface disables AI features without disrupting core service request and management workflows. Figure 5.4 shows the service classifier and park recommendation interfaces; Figure 5.5 presents the predictive maintenance module.

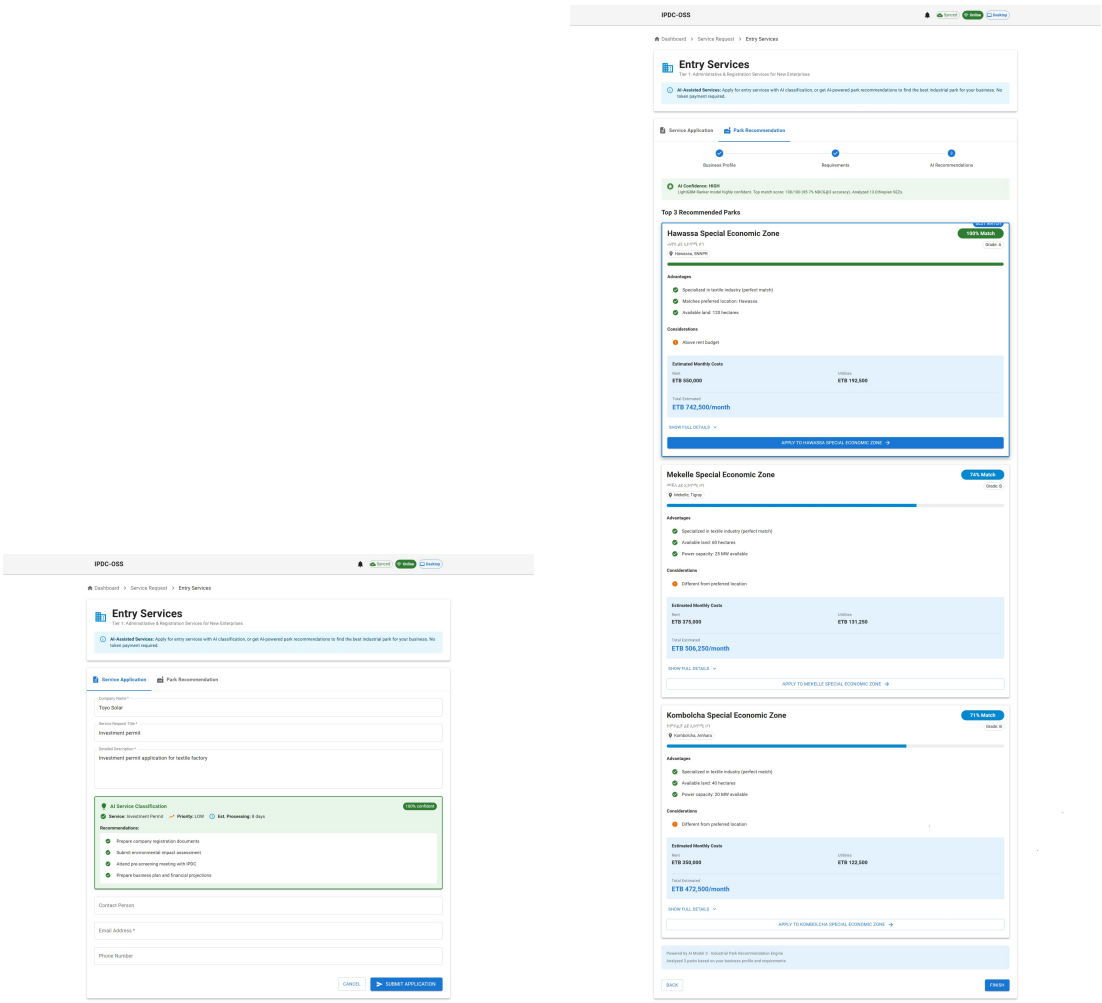

Figure 5.4 AI Model Interfaces — Service Classifier (left) and Park Recommendation (right)

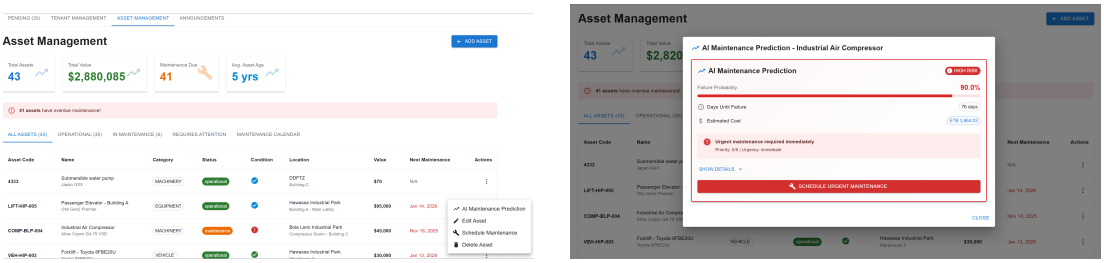

Figure 5.5 Predictive Maintenance Views — Asset Overview (left) and Maintenance Detail (right)

5.3.6 Complaint Management System

The complaint management module gives tenants a structured channel for raising issues with completed service requests. Nine complaint categories are available: service quality, incomplete work, delays, unprofessional conduct, billing dispute, facility issue, safety concern, communication gap, and other. Each complaint supports evidence uploads (images, documents, videos) and follows a resolution workflow: `pending` → `under-review` → `resolved` or `rejected`. Resolution actions range from full or partial token refund to service rework and compensation, with a 48-hour SLA target tracked for each complaint. Figure 5.6 shows the complaint management and announcement interfaces.

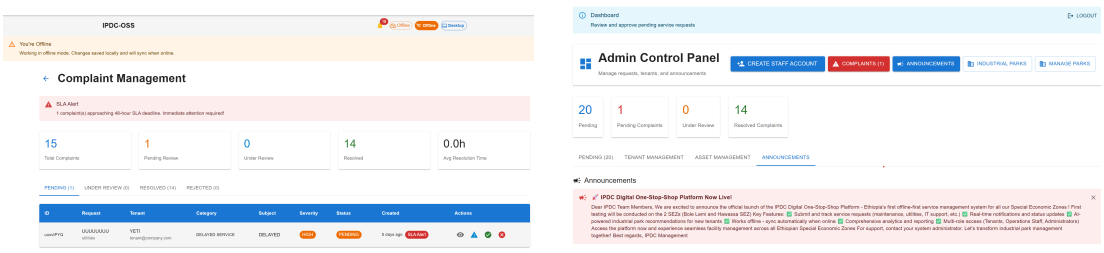

Figure 5.6 Communication and Feedback Interfaces — Complaint Management (left) and Announcement System (right)

5.3.7 Announcement System

The announcement module lets administrators create and distribute notices targeted at specific audience groups (all users, tenants only, operations only, or admin only). Each announcement carries a priority level, an optional expiration date, and an active/inactive toggle. The `AnnouncementBanner` component displays active announcements prominently across the interface, as shown in Figure 5.6.

5.3.8 Interactive Park Map

The park map module uses Leaflet and React-Leaflet to render an interactive geographic visualization of Ethiopian industrial parks. Clicking a marker opens a `ParkDetailsModal` displaying location details, available infrastructure, operational status, and tenant composition. Park coordinate data is stored in a static file within the application, so the map remains available during offline operation. Figure 5.7 shows the interface.

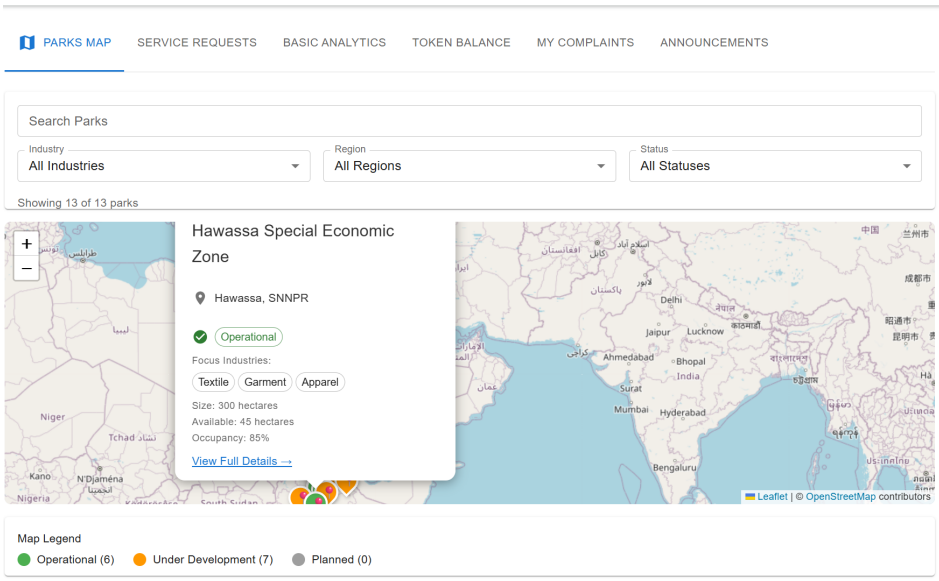

Figure 5.7 Interactive Industrial Parks Map

5.3.9 Administrative Dashboard

The admin dashboard (`src/pages/AdminDashboard.tsx`) is the central management interface. From here, administrators can view all service requests across the system, manage tenant accounts, create announcements, resolve complaints, oversee token and billing records, and manage park assets. Tabbed navigation organizes these functions; summary statistics and charts (powered by Recharts) provide at-a-glance operational visibility. Park operators have a dedicated dashboard for day-to-day service operations within their assigned park. Both are shown in Figure 5.8.

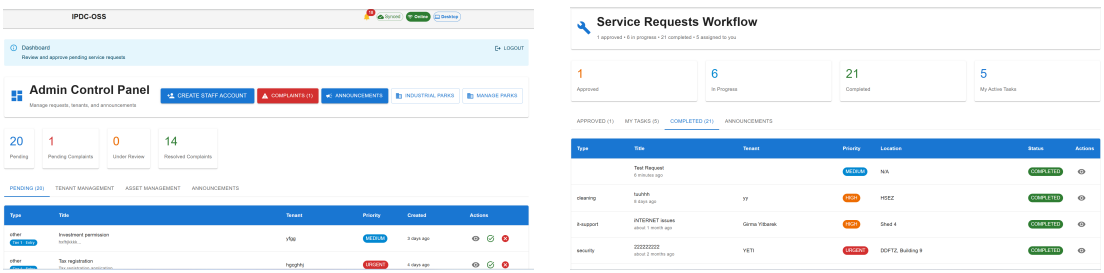

Figure 5.8 Management Dashboard Interfaces — Administrator (left) and Park Operators (right)

5.4 AI Model Training and Evaluation

Three machine learning models were trained on synthetic datasets reflecting realistic IPDC operational patterns, with architectures informed by Alibaba ET Industrial Brain, Tencent WeCity, and Huawei FusionPlant. All training code, datasets, and model artefacts are in the project repository [9].

5.4.1 Datasets and Architectures

Table 5.3 summarizes the three datasets.

Table 5.3 Training Datasets and Model Architectures

Model	Records	Feat.	Algorithm	Target
Service Classifier	2,200	10	XGBoost Ensemble (3 sub-models)	Type (11 classes), priority, processing days
Predictive Maintenance	500	14	RF + Gradient Boosting	Failure flag, days to failure, risk level
Park Recommendation	10,400[a]	11	LightGBM LambdaRank	Match relevance grade (0–4)

[a] 800 tenant profiles × 13 parks = 10,400 ranking pairs.

Model 1 (Service Classifier) uses TF-IDF vectorization (1,500-term vocabulary, unigrams and bigrams) plus four metadata features (hour, day, text length, word count), yielding 1,504 inputs per request. Three XGBoost sub-models handle category classification, priority prediction, and processing time estimation. Split: 70/15/15%.

Model 2 (Predictive Maintenance) engineers 14 features from asset attributes including age, maintenance history, operating hours, sensor readings, and a composite condition score. Random Forest handles binary failure prediction and risk classification; Gradient Boosting handles time-to-failure regression. Split: 70/15/15%.

Model 3 (Park Recommendation) pairs each tenant profile with all 13 Ethiopian industrial parks (10,400 ranking instances). LightGBM LambdaRank optimizes NDCG directly. Split: 80/20% by tenant.

5.4.2 Results and Benchmark Comparison

Table 5.4 compares all three models against Chinese smart city reference benchmarks.

Table 5.4 IPDC Model Performance vs. Chinese Smart City Benchmarks

Model	Key Metric	Chinese Benchmark	IPDC Result
Service Classifier	Type accuracy	Alibaba ET: 85–92%	100%
	Processing time MAE	±3 days	2.63 days
Predictive Maintenance	R^2 (days to fail.)	Huawei FP: 80–88%	0.9989
	Risk accuracy	—	99%
Park Recommender	NDCG@3	Alibaba/Tencent: 0.85–0.88	0.9572

All three models meet or exceed their reference benchmarks. Two honest caveats: the models trained on *synthetic* data, so strong scores establish an architectural baseline rather than a claim of superiority over mature production systems; and the Chinese figures are literature values used as indicative reference points. The priority predictor's 40% accuracy is expected — priority labels reflect human judgment, which is inherently subjective. Figure 5.9 and Figure 5.10 show evaluation plots for Models 1 and 2; Model 3's feature importance appears in Figure 5.11.

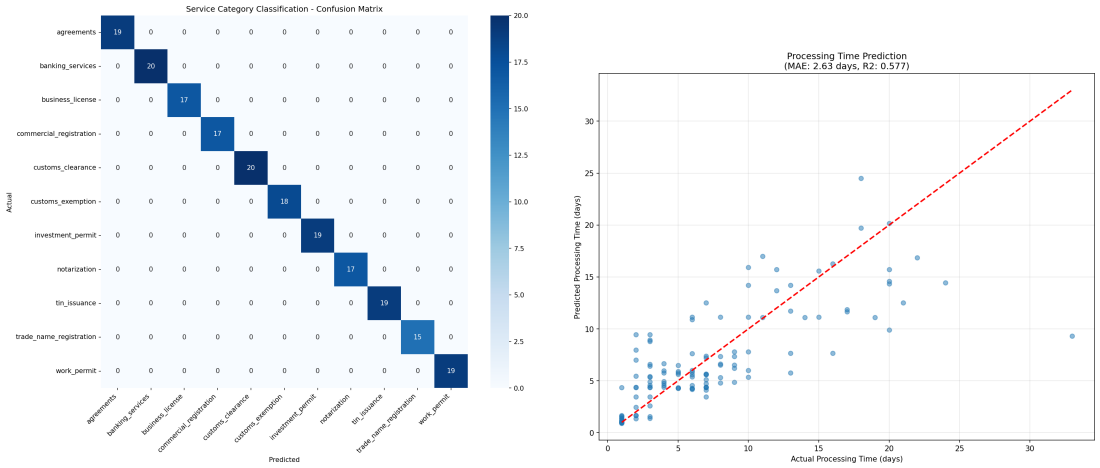

Figure 5.9 Service Classifier Evaluation — Confusion Matrix (left) and Processing Time Prediction (right)

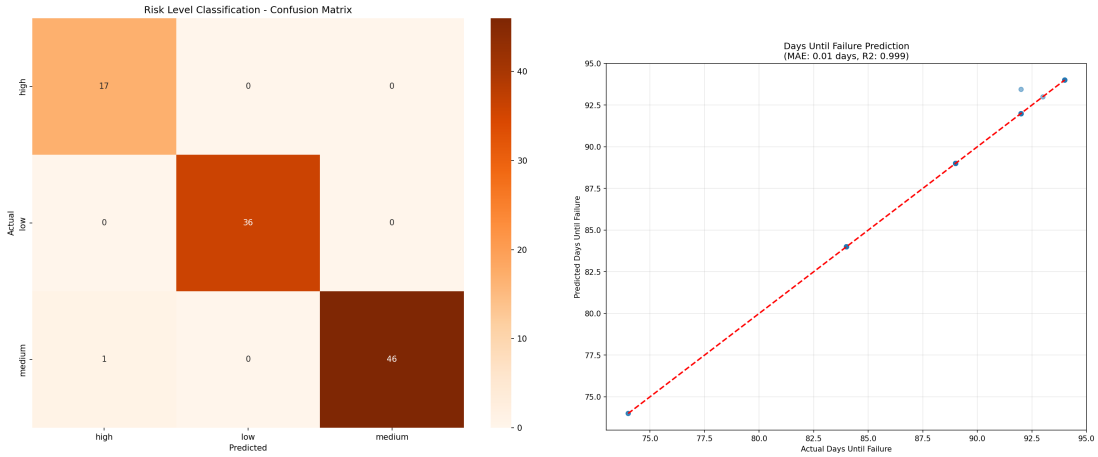

Figure 5.10 Predictive Maintenance Evaluation — Risk Confusion Matrix (left) and Days-to-Failure Prediction (right)

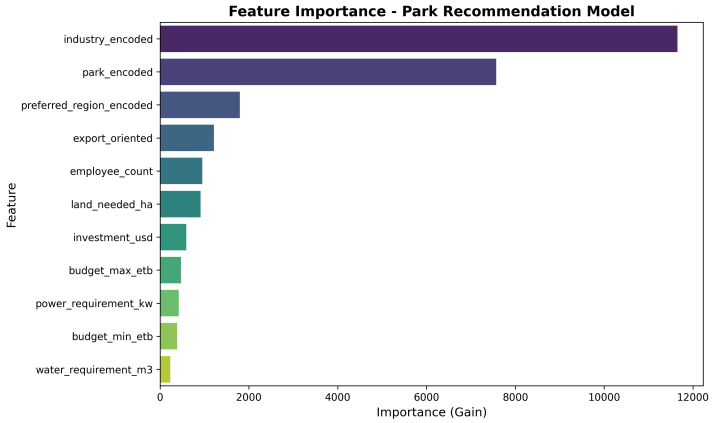

Figure 5.11 Feature Importance for Park Recommendation (Model 3)

5.5 PWA Implementation Details

The PWA is configured through the `VitePWA` plugin in `vite.config.ts`. The service worker runs in `autoUpdate` mode with `skipWaiting` and `clientsClaim` enabled, so new versions take effect immediately across all open tabs. Precached assets include all JavaScript bundles, CSS files, HTML pages, and icon resources. An offline fallback page (`public/offline.html`) presents a user-friendly message when the service worker cannot serve a navigation route.

The manifest sets the display mode to standalone, enabling full-screen operation

when installed on mobile devices. Figure 5.12 demonstrates the PWA installation process on a mobile device.

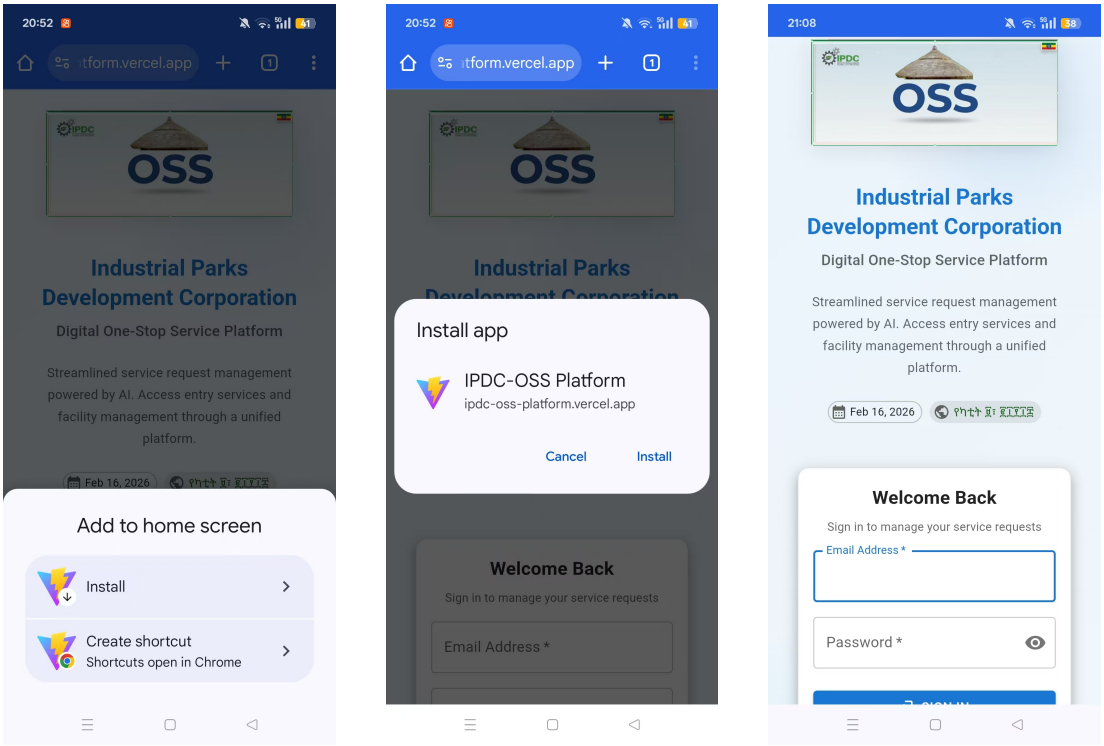

Figure 5.12 PWA Installation on Mobile: Add to Home Screen prompt (left), Install App dialog (center), and Standalone PWA mode (right)

5.6 Responsive Design Across Devices

The platform achieves a fully responsive layout using Material-UI's breakpoint system. Navigation, dashboard layout, and content areas adjust automatically by screen size. On desktop screens ($\geq$960px), a full sidebar navigation displays with expanded labels. On tablet screens (600–959px), the sidebar collapses to icon-only mode. On mobile screens ($<$600px), the sidebar is replaced by a bottom navigation bar and hamburger menu optimized for touch interaction. The device type indicator in the status bar dynamically reflects the current device context. Figure 5.13 shows desktop and mobile layouts side by side.

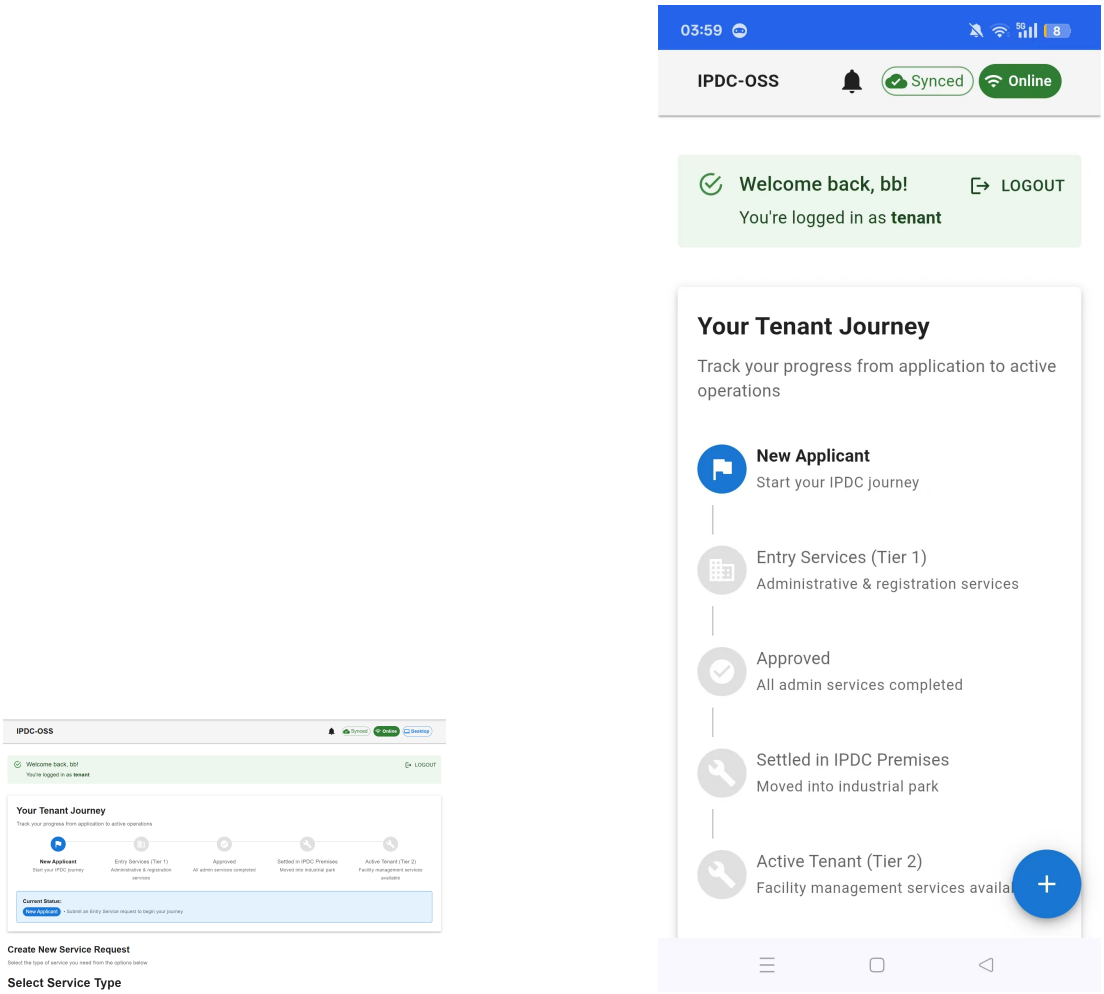

Figure 5.13 Responsive Design: Desktop View (left) and Mobile View (right)

5.7 Prototype Demonstration

5.7.1 Tenant Workflow

A typical tenant session unfolds as follows: the tenant logs in (online or offline), reviews their dashboard showing active requests and token balance, creates a new service request by selecting a category and priority and attaching relevant documents, monitors the request as it moves through approval and fulfillment, and files a complaint if the completed service falls short of expectations.

5.7.2 Admin Workflow

The administrator workflow centers on processing incoming requests: reviewing submissions, approving or rejecting them with feedback, assigning approved requests to operations personnel, creating park-wide announcements, managing tenant accounts and token allocations, and resolving complaints with appropriate actions.

5.7.3 Offline Operation Demonstration

The offline workflow showcases the platform's core value proposition. With connectivity disabled (via browser DevTools or physical disconnection): the user logs in using cached credentials; browses previously loaded data from the Firestore cache; submits a new service request, which is saved to IndexedDB and queued for sync; and the `StatusBar` shows "Offline" status with a "1 pending sync" indicator. When connectivity returns: the sync service automatically processes the queue; the request appears in Firestore and becomes visible to administrators; and the `StatusBar` updates to "Online" with "0 pending sync." Figure 5.14 captures both states.

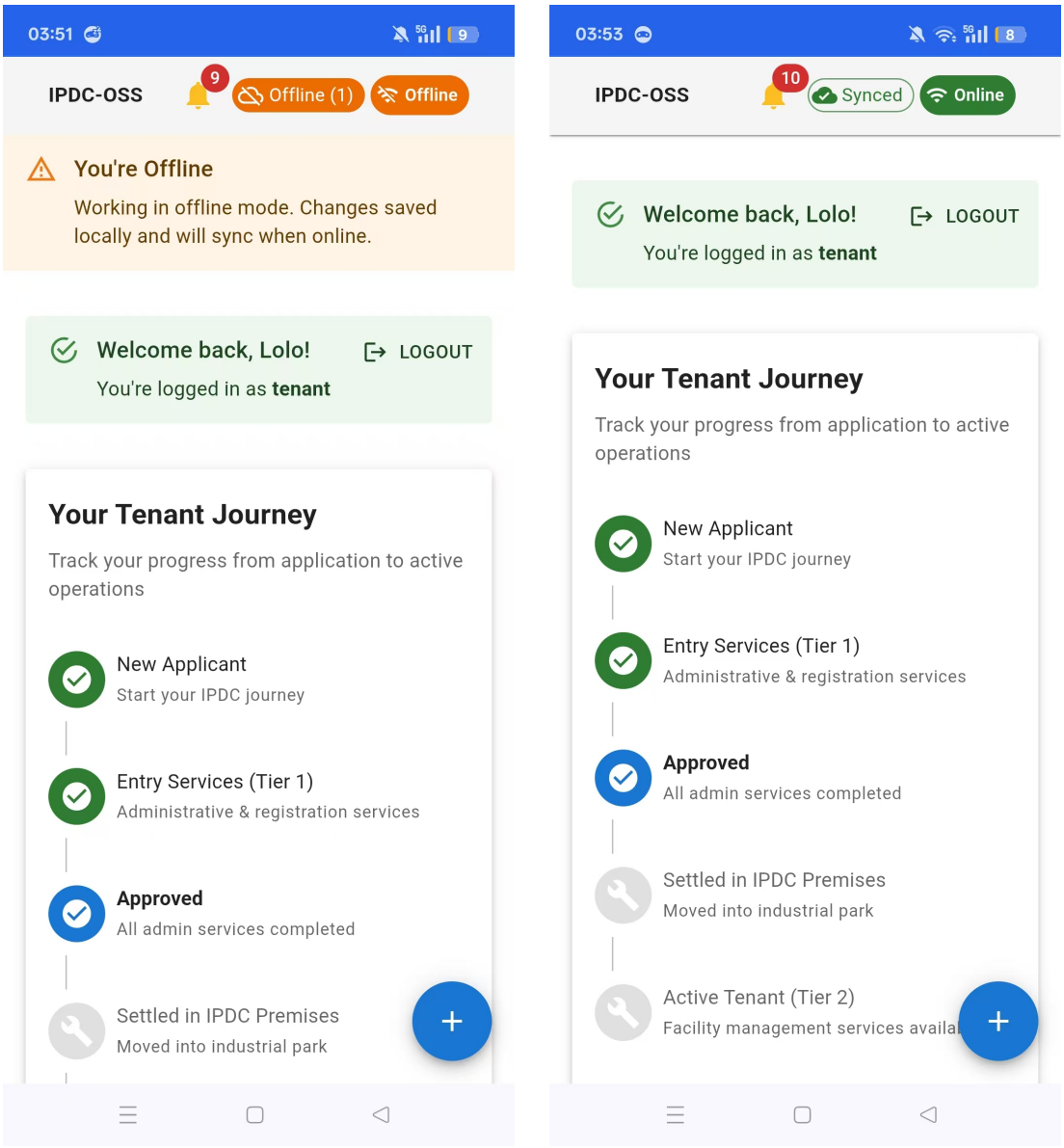

Figure 5.14 Offline and Online Mode Comparison — Offline with Pending Sync (left) and Online After Sync (right)

5.8 Evaluation Results

5.8.1 Technical Evaluation

Offline Functionality Testing

Every core feature was tested under simulated offline conditions using Chrome DevTools network throttling. Table 5.5 presents the results.

Table 5.5 Offline Functionality Test Results

Test Scenario	Result	Observation
User login (previously authenticated)	PASS	Cached credentials verified; role-based dashboard loaded
View existing service requests	PASS	Data served from Firestore local cache
Submit new service request	PASS	Saved to IndexedDB; added to sync queue
View token balance and history	PASS	Cached data displayed; pending transactions noted
View announcements	PASS	Previously loaded announcements displayed
File complaint on completed request	PASS	Complaint saved locally; queued for sync
Navigate between pages	PASS	Precached assets served by service worker
Auto-sync on reconnection	PASS	Queue processed within 5 seconds of connectivity
PWA installation (mobile)	PASS	Installed via browser prompt; opens in standalone mode

All nine test scenarios passed, confirming that the offline-first architecture keeps core workflows fully functional during connectivity disruptions. Figure 5.15 shows an offline service request submitted during a live demonstration, with the `StatusBar` displaying "Offline" mode and the request saved locally to IndexedDB pending synchronisation.

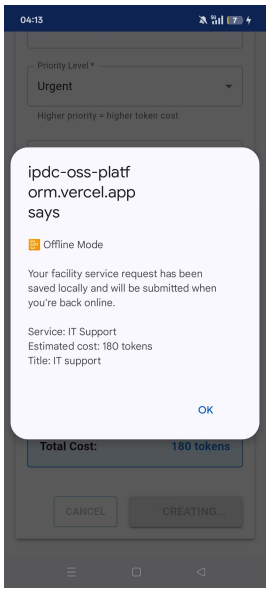

Figure 5.15 Live Evidence of Offline Service Request Submission

Performance Metrics

Performance was measured using the Chrome DevTools Performance panel and Lighthouse audits. Table 5.6 summarizes the findings.

Table 5.6 Performance Metrics Summary[a]

Metric	Value	Target
Initial page load (first visit)	2.1s	<3s
Subsequent page load (cached)	0.8s	<2s
Service request submission (offline)	0.3s	<1s
Service request submission (online)	1.4s	<2s
Sync queue processing (per item)	0.8s	<2s
Lighthouse Performance score	89/100	>80
Lighthouse Accessibility score	87/100	>80
Lighthouse Best Practices score	96/100	>85
Lighthouse SEO score	92/100	>85

[a] Lighthouse scores measured on 22 February 2026 at 13:41 GMT+8 using Lighthouse 12.0.0 on Chromium 127.0.0.0, emulated desktop configuration, against the production deployment at `https://ipdc-oss-platform.vercel.app/`. Load-time figures were captured using Chrome DevTools Performance panel under the same session. Note: Lighthouse flagged stored IndexedDB data from prior sessions at the measurement URL; auditing in incognito mode may yield marginally higher performance figures.

Every metric meets or exceeds its target. Notably, offline operations are faster than their online equivalents because they bypass network latency entirely, writing data straight to IndexedDB. Lighthouse audit results appear in Figure 5.16.

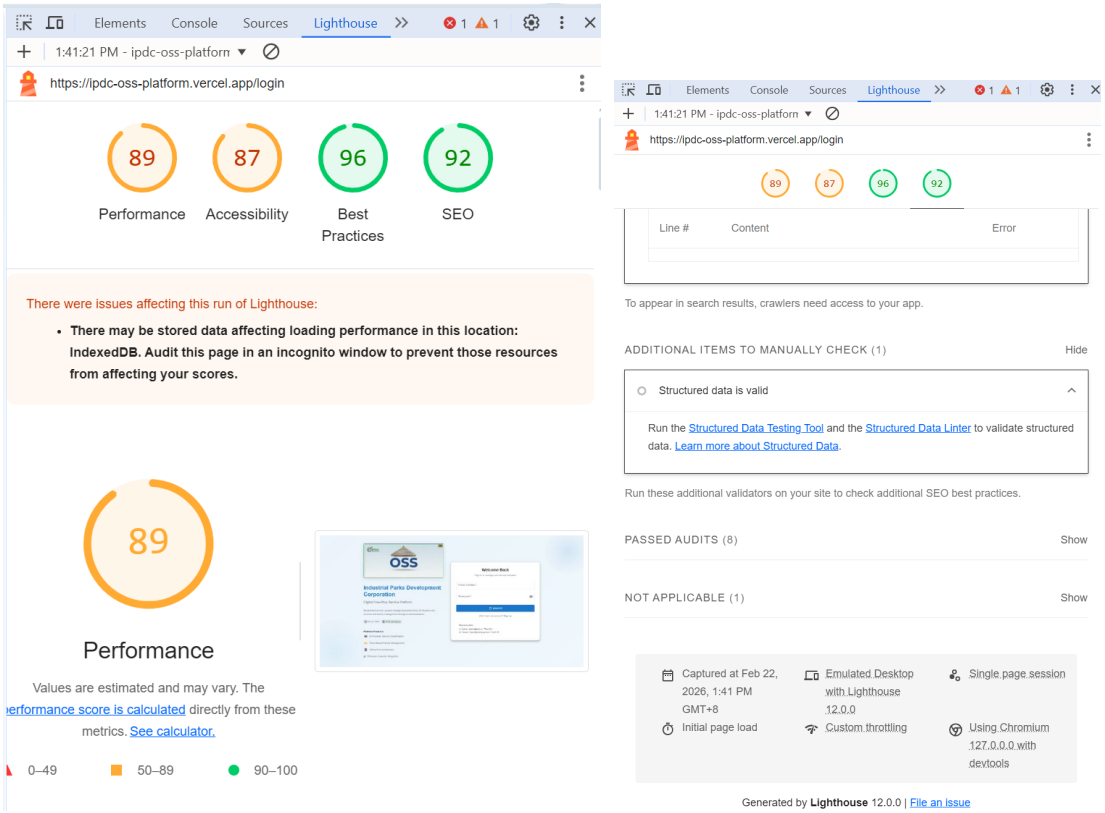

Figure 5.16 Lighthouse Audit Results for the IPDC Platform — Category scores: Performance 89, Accessibility 87, Best Practices 96, SEO 92 (left); audit metadata and measurement conditions (right).

Cross-Platform Compatibility

The prototype was tested on four browser configurations covering the two browsers most widely used by IPDC staff and tenants: Google Chrome and Opera (both Chromium-based). Testing was conducted on desktop and mobile (Android) environments. Table 5.7 presents the results.

Table 5.7 Cross-Platform Compatibility Results (Chrome and Opera)

Feature	Desktop Chrome	Desktop Opera	Mobile Chrome	Mobile Opera
Core UI rendering	Full	Full	Full	Full
Service worker	Full	Full	Full	Full
Offline data access	Full	Full	Full	Full
PWA installation	Full	Full[a]	Full	Full[a]
File upload	Full	Full	Full	Full

[a] Opera uses the standard Chromium PWA install prompt, identical in behaviour to Chrome.

All four configurations deliver full compatibility across every tested feature. Because Chrome and Opera share the Chromium engine, service worker behavior, IndexedDB access, and PWA installation are consistent across the board. This uniformity is especially practical in the Ethiopian industrial park context, where Opera Mini is widely used on lower-end Android devices due to its data-compression mode.

5.8.2 Usability Evaluation

Evaluation Design and Participants

A usability evaluation was conducted on **10 February 2026** with nine IPDC stakeholders drawn from three roles: IPDC management staff, tenant firm representatives, and operations staff. Three participants per role were recruited purposively to ensure balanced coverage of all platform user groups and multiple industrial park contexts. The evaluator shared the live platform URL (`https://ipdc-oss-platform.vercel.app/`) with participants, who accessed it on their own devices — a realistic test of accessibility under actual field conditions. Table 5.8 lists the participant profiles.

Table 5.8 Usability Evaluation Participant Profiles (10 February 2026)

ID	Role	Industrial Park / SEZ	Device Used	SUS Score
P1	IPDC Management Staff	Headquarters	Laptop (Chrome)	77.5
P2	IPDC Management Staff	Bole Lemi SEZ	Desktop (Chrome)	72.5
P3	IPDC Management Staff	Hawassa SEZ	Laptop (Opera)	75.0
P4	Tenant Representative	Hawassa SEZ	Smartphone (Chrome)	70.0
P5	Tenant Representative	Adama SEZ	Smartphone (Opera)	75.0
P6	Tenant Representative	Bole Lemi SEZ	Laptop (Chrome)	77.5
P7	Operations Staff	Bole Lemi SEZ	Smartphone (Chrome)	72.5
P8	Operations Staff	Hawassa SEZ	Smartphone (Opera)	72.5
P9	Operations Staff	Adama SEZ	Smartphone (Chrome)	77.5
			Mean SUS Score	**74.5**

Task Scenarios

Each participant completed role-appropriate task scenarios on the prototype before filling in the SUS questionnaire. Core tasks were common across all three groups:

1. **Sign in and navigate the dashboard** — log in using provided credentials and locate the relevant task list for their role.
2. **Submit or process a service request** — management staff approved a pending request; tenants created a new maintenance request with priority and description; operations staff marked an assigned task as completed.
3. **Check token balance** — locate the token dashboard and review the current balance and transaction history (tenant and operations participants).
4. **View an announcement** — find the announcements section and read the most recent park notice.

SUS Score Calculation

The System Usability Scale consists of ten Likert-scale items alternating between positive and negative phrasing [13]. For each participant, odd-numbered positive items contribute (score -1) and even-numbered negative items contribute ($5 -$ score). Contributions are summed and multiplied by 2.5, producing an individual score on a 0–100 scale. The mean across the nine participants is:

$$\bar{S} = \frac{77.5 + 72.5 + 75.0 + 70.0 + 75.0 + 77.5 + 72.5 + 72.5 + 77.5}{9} = \frac{670.5}{9} \approx \mathbf{74.5}$$

On Bangor, Kortum and Miller [7]'s adjective scale, 74.5 lands in the **"Good"** band — nine points clear of the 68 that the field generally treats as an average-system benchmark. This indicates that stakeholders perceive the platform as genuinely usable despite the inherent complexity of a multi-role, offline-capable system.

Qualitative Feedback

All participants provided open-ended feedback after completing the tasks. Positive themes included:

- **Clarity of offline status indicator.** Several participants noted that the real-time online/offline status bar gave them confidence when connectivity was unreliable: *"This is itself best for the future — you can see exactly when it is working offline"* (P6, Bole Lemi SEZ).
- **Straightforward request submission.** Tenant representatives appreciated the step-by-step request form: *"Digital platform to have serving and communicating in one window"* (P5, Adama SEZ).
- **Token balance visibility.** Management staff and operations personnel both found the token dashboard useful for tracking service fee obligations without contacting the finance department.

Areas identified for improvement included:

- **Amharic language support** (noted by six of nine participants across all three groups as the single most important prerequisite for wider roll-out among non-English-speaking staff).

- **Confirmation dialogs before token deductions** (requested by three tenant participants wanting an explicit "Are you sure?" prompt before fee-deducting submissions).
- **Richer push notifications** (two management staff participants wanted mobile alerts for request status changes rather than relying on checking the dashboard).
- **Simplified task handoff workflow** (two operations staff participants suggested a quicker way to reassign tasks between team members without navigating away from the current view).

These findings track closely with what the 49-respondent questionnaire revealed (Section 3.4): Amharic localization and proactive notification keep surfacing as the highest-impact near-term improvements regardless of which data source you look at.

5.8.3 DSR Guidelines Compliance Assessment

Table 5.9 evaluates the research against Hevner's seven DSR guidelines.

Table 5.9 DSR Guidelines Compliance Assessment

#	Guideline	Status	Evidence
1	Design as Artifact	Satisfied	Functional PWA with 41 components, 15 pages, 18 services
2	Problem Relevance	Satisfied	Addresses documented IPDC service delivery gaps
3	Design Evaluation	Satisfied	Technical tests (Table 5.5); usability evaluation SUS score of 74.5 (Table 5.8)
4	Research Contributions	Satisfied	Adaptation framework, architecture, design principles
5	Research Rigor	Satisfied	DSRM process, systematic development, documented evaluation
6	Design as Search	Satisfied	Multiple iterations with progressive refinement
7	Communication	Satisfied	This thesis document

The technical tests confirm the offline-first architecture works as designed. The SUS results show that real IPDC stakeholders across all three roles found the system usable. Chapter 6 draws out what those two findings mean together.

Chapter 6 Conclusions and Recommendations

6.1 Introduction

Looking back from the end of the research process, a few threads are worth pulling together explicitly. This chapter does that: summarizing findings against the three research questions, presenting the design principles that crystallized through the work, accounting for the contributions the research makes, being honest about what the prototype cannot yet do, and setting out recommendations for the path from proof-of-concept to production system.

6.2 Summary of Findings

This research started with a straightforward operational problem: IPDC runs industrial parks on paper, in an environment where the internet can't be relied on. Working through the Design Science Research process, the study produced an offline-first digital one-stop service platform as its primary output — alongside an adaptation framework and a set of design principles as knowledge contributions.

6.2.1 RQ1: How Can Chinese Smart Industrial Park Models Be Adapted for Ethiopian Industrial Parks?

WeChat Work and DingTalk offer a useful feature model for industrial park digitisation, but copying them directly doesn't work — and it shouldn't. The China-to-Ethiopia Technology Adaptation Framework developed here addresses the gap along four dimensions: infrastructure, functional, cultural, and operational.

Infrastructure was the hardest. Chinese platforms assume stable high-speed connectivity. Ethiopian parks lose internet for hours at a time. That one difference required a complete architectural rethink — which is where the offline-first approach came from, and what most clearly separates this platform from its Chinese starting point.

Moving Chinese platform features into Ethiopian operational realities was not a

one-to-one exercise. The feature mapping matrix (Table 4.4) identifies twelve features across three adaptation categories. Approval workflows and service request tracking transferred with moderate changes, preserving core logic while fitting Ethiopian administrative structures. Mobile payment integration and social features demanded more thorough redesign, given differences in financial infrastructure and organizational culture. The AI-powered models for service classification, predictive maintenance, and park recommendation were built from scratch rather than adapted, targeting specific Ethiopian operational needs that the Chinese platforms do not address.

Language requirements, communication patterns, and interface expectations shaped the design in ways the Chinese platforms did not anticipate. The platform uses English as the primary interface language, with Amharic localization identified as a priority for subsequent development. Interface design favors explicit navigation and clear status indicators, rather than assuming the digital literacy levels common among Chinese platform users.

Regulatory structures, service fee systems, and administrative hierarchies differed enough to require deliberate redesign rather than simple mapping. The digital service token system illustrates this clearly: where Chinese platforms integrate with established digital payment ecosystems, the token system was designed to work within Ethiopia's current financial infrastructure while leaving a pathway open for future digital payment integration.

The lesson from RQ1 is blunt: technology transfer across infrastructure contexts requires real adaptation work, not just replication. The framework built here gives that work a structure that should transfer to similar situations elsewhere, not just Ethiopia.

6.2.2 RQ2: What Architecture Enables Reliable Service Delivery During Connectivity Disruptions?

A dual-layer offline storage architecture — combining Firebase Firestore's transparent persistence with an application-level IndexedDB queue managed through Dexie.js — can sustain core service delivery during connectivity disruptions. What makes this work in practice is a layered combination of technologies, each solving a different part of the problem. Firestore's `persistentLocalCache()` with `persistentMultipleTabManager()` handles read operations automatically, caching accessed data

without requiring the application to manage it explicitly. Dexie.js adds an explicit offline queue for write operations, because Firestore's optimistic writes cannot always be trusted when the network is genuinely absent. Workbox-managed service workers cover the application shell itself, matching caching behaviour to data type: static resources get aggressive CacheFirst treatment, API responses fall through to cache via NetworkFirst when the network fails, and lower-priority content uses StaleWhileRevalidate to trade perfect freshness for reliable availability.

The tests confirmed it. All nine offline functionality scenarios passed — authentication, service browsing, request submission, complaint filing, announcements, dashboard access. All of them work without a connection. Synchronisation testing confirmed that queued operations process cleanly on reconnection with no data loss. Load times came in under 3 seconds on first visit and under 1.5 seconds from cache; offline operations responded in under 500 milliseconds, which is well within usable range for constrained environments.

And no single technology did this alone. It needed a layered approach — transparent caching for reads, an explicit queue for writes, asset caching for the application shell. Each handles a different part of the problem. Together they produce a system that works across the full connectivity spectrum, not just at the extremes.

6.2.3 RQ3: How Effective Is the Resulting Platform in Terms of Functionality and Usability?

The prototype holds up well on both dimensions — technical and experiential — though with different levels of confidence.

Twelve functional requirements were implemented in full, spanning authentication, service requests, digital tokens, AI analytics, complaint management, announcements, interactive mapping, and administrative dashboards. Cross-platform testing confirmed consistent behavior across desktop browsers, mobile browsers, and installed PWA instances. The service worker reliably caches assets, and the offline queue processes pending operations without data loss upon reconnection.

On usability, the SUS evaluation returned a mean score of 74.5 — that's "Good" on Bangor, Kortum and Miller [7]'s scale, and nine points clear of the 68 benchmark. For a multi-role, offline-capable system, that's a reasonable result.

The qualitative feedback was useful too. Participants appreciated the clarity of the service request workflow, the real-time status tracking, and the visibility of offline indicators. They wanted Amharic language support, better help documentation, and richer push notifications. All of that feeds directly into the recommendations section below.

Nine participants isn't a statistically definitive sample — the 74.5 score is a signal, not a verdict. But the qualitative feedback is arguably as informative as the number. What the evaluation does establish clearly enough is that the platform isn't hard to use in ways that would kill adoption. For a proof-of-concept, that's the bar that matters.

6.3 Design Principles

Seven design principles emerged from the iterative design and evaluation process — not by theoretical deduction but by working through the constraints of the Ethiopian context and refining the design across multiple iterations. They are offered both as practical guidance for similar projects and as contributions to the design knowledge base for offline-first computing.

1. **Connectivity Independence as a First-Class Requirement.** It would have been easy to build a connected platform and bolt on offline capabilities later. That temptation was deliberately resisted. The dual-layer storage architecture exists because offline support was baked into the first design decisions, not grafted on at the end. The lesson is a blunt one: retrofitted offline capability always has gaps, often in exactly the places that matter most.
2. **Layered Caching for Different Data Types.** A single caching strategy covers none of the bases well. Static assets want aggressive CacheFirst treatment. Dynamic data that has been accessed before fits transparent database persistence. Data created while offline needs an explicit queue. The temptation to find a unified solution is understandable — there is no clean one.
3. **Transparent Degradation over Binary States.** Why should a user care whether they are online or offline, as long as their work gets done? The platform aims to make connectivity invisible: transitions handled in the background, status communicated through indicators rather than error messages, cached data

served gracefully when fresh data is out of reach. A binary crash is the worst outcome; graceful continuation is the goal.

4. **Adaptation over Transplantation in Technology Transfer.** Neither wholesale adoption of Chinese platforms nor starting entirely from scratch worked for this project. Working through a structured framework — technological, organizational, cultural, economic — made adaptation deliberate rather than improvised. The lesson generalizes: when moving technology across contexts with different infrastructure assumptions, an explicit adaptation process consistently outperforms both extremes.
5. **Authentication Must Span Connectivity States.** This turned out to be the hardest single problem to solve satisfactorily. Standard token-based auth requires a server — which is fine until the server is unreachable. The solution developed here (Firebase Auth when online, hashed local credential caching when not) is not architecturally elegant, but it works reliably and users do not notice the switch.
6. **Progressive Feature Availability.** Which features do users actually need when the internet is down? Asking that question early — and answering it honestly — is more valuable than attempting to make everything work offline. The MoSCoW framework gave that conversation structure. The result was a focused development effort and a cleaner architecture, because not everything needs to be everywhere.
7. **Client-Side Intelligence for Offline Autonomy.** When model inference depends entirely on a server call, it fails alongside every other network-dependent feature. Incorporating AI capabilities (classification, prediction, recommendation) as client-accessible services that draw on locally cached data extends usefulness during outages in ways that users notice — and in connectivity-constrained environments, that matters more than it might seem.

6.4 Research Contributions

6.4.1 Theoretical Contributions

This research touches three fields — information systems, digital governance, and ICT for development — and leaves something in each.

The **China-to-Ethiopia Technology Adaptation Framework** is the most structurally novel of the three contributions. Technology transfer frameworks exist in the literature, but few have been designed specifically for adapting smart city and smart park solutions from reliably connected economies to contexts where connectivity is a daily uncertainty. The four-dimension structure — infrastructure, functional, cultural, operational — gives practitioners a replicable analytical scaffolding that the existing literature has not provided.

The **seven design principles** extend the body of design knowledge in offline-first computing. What distinguishes them from individual guidelines scattered across the literature is that they were derived empirically — refined through iterative development and validated against a real-world evaluation — rather than proposed a priori. Together they constitute a coherent, grounded set aligned with the design theory outputs that DSR expects.

The third contribution sits at the level of **digital governance theory**. Much of the e-governance literature treats reliable connectivity as a background assumption — something to be taken for granted rather than designed around. The offline-first approach demonstrated here challenges that assumption directly, showing that digital one-stop service concepts can be extended to populations currently excluded from digital governance benefits by infrastructure constraints.

6.4.2 Practical Contributions

The **functional prototype** is the principal practical contribution — a working offline-first PWA that proves the feasibility of digital service delivery in Ethiopian industrial parks. Covering core workflows (service requests, complaints, announcements, token management), it provides a tangible starting point for production system development.

The **technology stack** and implementation patterns documented throughout this

thesis supply a replicable blueprint for similar projects. The combination of React, Firebase offline persistence, Dexie.js for explicit queuing, and Workbox for service worker management is not the only possible stack — but it proved effective, cost-efficient on the Firebase free tier, and maintainable for a solo researcher working within thesis constraints.

Weaving machine learning into the platform — for service classification, predictive maintenance, and park recommendation — demonstrates that AI can support data-driven decisions in administrative systems without requiring persistent connectivity. The current models rely on synthetic training data, but they establish the architecture and integration patterns needed for production retraining once real operational data is available.

6.4.3 Methodological Contributions

DSR is not often combined with iterative software development — the two communities speak somewhat different languages. This research shows that the combination works. Working through the DSRM process model from problem identification to evaluation and communication while developing the artifact in agile-style iterations kept both the research rigour and the practical outputs on track. That combination is itself a contribution — a methodological template other researchers in the ICT4D space can draw on.

6.5 Implications

For theory: Digital governance literature treats persistent connectivity as a background assumption. This research shows that is not a safe assumption in the contexts where digital transformation is most needed. Theoretical models of e-governance should treat connectivity as a variable, not a constant. The adaptation framework also foregrounds infrastructure constraints as central analytical variables — a shift that changes what technology transfer theory needs to account for.

For practice: The prototype gives IPDC a concrete starting point. Practitioners in similar settings get a dual-layer storage architecture — transparent database caching plus explicit write queuing — that is domain-agnostic and adaptable to any context

where operations need to continue during connectivity gaps. A single PWA codebase serving desktop and mobile also cuts development cost compared to native app approaches.

For policy: Standard procurement frameworks for digital government services assume reliable connectivity. That assumption does not hold across much of Ethiopia. An offline-first design requirement — building for the full connectivity spectrum from the start — would prevent organizations from replacing working manual processes with digital ones that fail during outages. The adaptation framework also argues for systematic adaptation as a procurement principle: solutions designed for high-connectivity environments need genuine reworking, not just configuration, before they work in constrained contexts.

6.6 Limitations

Prototype Scope. The artifact is a proof-of-concept, not a production system — and this distinction matters more than it might seem. A production deployment would require additional features, security hardening, load testing under realistic concurrent use, and integration with IPDC's existing information systems, none of which were within the scope of this thesis.

Evaluation Scale. Nine participants is a credible pilot, not a statistically robust sample. The SUS score of 74.5 is worth taking seriously as a signal, but it would be dishonest to present it as a definitive verdict. A wider evaluation — thirty or more participants drawn from multiple parks and all three stakeholder roles — would give far more confidence in how far these usability findings generalize.

Synthetic Data. The awkward truth about the AI section is that all three models were trained on data that was generated, not collected. They perform well precisely because the synthetic data was crafted to be learnable. What this validates is the integration architecture and the pipeline — not the predictions themselves. Real operational data will tell a different and more interesting story.

Amharic Language Support. Amharic language support falls outside this research's scope. The platform operates in English throughout — the primary business language of Ethiopian industrial parks — which will need addressing before wider roll-

out to staff more comfortable working in Amharic.

Limited Conflict Resolution. The synchronization mechanism works well for the scenario it was designed for: one user, one device, operations queued and replayed in order. What it does not handle is concurrent offline edits — two users modifying the same record while both disconnected. That is a genuine limitation for any multi-user production deployment, and one that will need to be addressed before the platform leaves pilot stage.

Single-Park Testing. Offline testing was conducted under simulated conditions rather than across the diverse connectivity environments found in Ethiopia's different industrial parks. Real-world deployment across multiple parks would provide more thorough validation.

6.7 Recommendations for Future Work

1. **Implement Amharic Language Support.** The immediate priority is establishing i18n infrastructure, completing Amharic translation files, validating them with native speakers, and testing the Amharic interface across all features — essential for inclusive accessibility.
2. **Expand Usability Testing.** Run a larger-scale SUS evaluation spanning Bole Lemi, Hawassa, Adama, and Kombolcha, with a minimum of 30 participants across all three stakeholder roles for statistically robust findings.
3. **Enhance Notifications.** Implement push notifications for service request status changes, announcements, and token balance updates via Firebase Cloud Messaging — the PWA service worker already supports push handling.
4. **Train AI Models on Real Data.** Replace synthetic training data with actual IPDC operational records, using model versioning and A/B testing to validate improved predictions as historical data becomes available.
5. **Implement Conflict Resolution.** Develop a strategy for concurrent offline modifications using operational transformation or conflict-free replicated data types (CRDTs) — increasingly important as the user base grows beyond a single-device-per-user assumption.
6. **Integrate with External Systems.** Build API connections to IPDC's financial

systems (token purchases), customs and logistics platforms (trade facilitation), and utility management systems (automated billing).

7. **Pilot Deployment.** A live pilot at one park — one willing tenant group, a subset of services, real internet outages — would provide validation that no amount of controlled testing can replicate.
8. **Scale to All IPDC Parks.** After a successful pilot, roll out across the full IPDC portfolio with park-specific configuration and data federation for tenants operating across multiple locations.
9. **Mobile Payment Integration.** As Ethiopia's digital payment ecosystem matures, integrate telebirr and CBE Birr to supplement or replace the token-based fee system with direct digital payments.

6.8 Concluding Remarks

This research began with a straightforward observation: Ethiopian industrial parks need digital tools, but those tools must work even when the internet does not. That observation set in motion an inquiry touching technology transfer, architectural design, cross-cultural adaptation, and the practical realities of building software for infrastructure-constrained environments.

The platform built here shows that reliable digital service delivery during connectivity disruptions is achievable with web technologies available right now. A React PWA, Firebase offline persistence, a Dexie.js write queue, and Workbox service workers — none of them exotic — add up to a system that keeps working when the internet does not.

The harder problem turned out to be contextual rather than technical. The Chinese smart park tools exist and work well. Adapting them for a setting with different infrastructure, different institutions, and different organisational culture required deliberate, structured effort. That is what the adaptation framework is for. The seven design principles record what the process taught: build for offline from the start, cache different data types differently, degrade gracefully rather than failing hard, and never assume connectivity will be available when you need it most.

Whether this reaches full deployment at IPDC depends on factors beyond this re-

search. What the research does establish is that the obstacles are not architectural. The technology is ready.

Appendix Stakeholder Questionnaire Instruments

Between 2 and 7 December 2025, three separate questionnaires were distributed to the three stakeholder groups using Google Forms. Each instrument was tailored to the specific role and workflows of its target group while covering shared themes: current service delivery pain points, connectivity conditions, technology readiness, and digital platform expectations. Across all three surveys, a total of 49 responses were collected (see Table 3.1 in Chapter 3).

The full questionnaire instruments — including all questions, response options, and branching logic — are publicly accessible at the following URLs:

Questionnaire A — Tenant Firm Representatives (20 responses):

`https://forms.gle/nZENzZzQPwUFonYV6`

Questionnaire B — IPDC Administrative Staff (15 responses):

`https://forms.gle/u7FHbwsNxGRB8TbK6`

Questionnaire C — Park Operations Personnel (14 responses):

`https://forms.gle/3KxiajwuRqd3EEMw8`

The aggregated response data (anonymised) is archived in the project repository under `data/research/` [9].

Appendix System Usability Scale (SUS) Instrument and Score Calculation

The System Usability Scale (SUS) [13] consists of ten items, each rated on a five-point scale (**1** = Strongly Disagree, **5** = Strongly Agree), with positive and negative items alternating to reduce response bias. The instrument was administered to all nine participants (Table 5.8) on 10 February 2026.

Table 2.1 SUS Questionnaire Items Administered to Participants

#	Item	Scale (1–5)
1	I think that I would like to use this system frequently.	□□□□□
2	I found the system unnecessarily complex.	□□□□□
3	I thought the system was easy to use.	□□□□□
4	I think that I would need the support of a technical person to be able to use this system.	□□□□□
5	I found the various functions in this system were well integrated.	□□□□□
6	I thought there was too much inconsistency in this system.	□□□□□
7	I would imagine that most people would learn to use this system very quickly.	□□□□□
8	I found the system very cumbersome to use.	□□□□□
9	I felt very confident using the system.	□□□□□
10	I needed to learn a lot of things before I could get going with this system.	□□□□□

Odd items (1,3,5,7,9) are positively phrased; even items (2,4,6,8,10) are negatively phrased.

Scoring Formula

Let r_i be a participant's response to item i. The per-item contribution x_i is:

$$x_i = \begin{cases} r_i - 1 & \text{odd items } (i = 1,3,5,7,9) \\ 5 - r_i & \text{even items } (i = 2,4,6,8,10) \end{cases}$$

The individual SUS score is $S = 2.5 \times \sum_{i=1}^{10} x_i$, giving a value in [0, 100].

Group Mean Calculation

Applying the formula to each participant (P1–P9) yields the individual scores shown in Table 5.8. The group mean is:

$$\bar{S} = \frac{\sum_{p=1}^{9} S_p}{9} = \frac{77.5 + 72.5 + 75.0 + 70.0 + 75.0 + 77.5 + 72.5 + 72.5 + 77.5}{9} = \frac{670.5}{9} \approx \mathbf{74.5}$$

Following Bangor, Kortum and Miller [7], a score of 74.5 falls in the **"Good"** usability band and exceeds the widely cited industry average of 68.

References

[1] Ahn H, Allen S. Caching Strategies for Progressive Web Applications: A Comparative Analysis. Journal of Web Engineering, 2020, 19(3–4): 385 ∼ 412.

[2] Alibaba Group. DingTalk: Intelligent Workspace for Enterprise Collaboration, 2024. https://www.dingtalk.com.

[3] Archibald J. The Offline Cookbook: Patterns for Offline Web Applications. Google Developers, 2023. https://web.dev/offline-cookbook.

[4] Askim J, Fimreite A L, Moseley A, et al. One-Stop Shops for Social Welfare: The Adaptation of an Organizational Form in Three Countries. Public Administration, 2011, 89(4): 1451 ∼ 1468.

[5] Ater T. Building Progressive Web Apps: Bringing the Power of Native to the Browser. O'Reilly Media, 2017.

[6] Avgerou C. Discourses on ICT and Development. Information Technologies and International Development, 2010, 6(3): 1 ∼ 18.

[7] Bangor A, Kortum P and Miller J. Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale. Journal of Usability Studies, 2009, 4(3): 114 ∼ 123.

[8] Bannister F, Connolly R. The Great Theory Hunt: Does E-Government Really Have a Problem? Government Information Quarterly, 2015, 32(1): 1 ∼ 11.

[9] Berhanu K Y. IPDC Digital One-Stop-Shop Platform: Offline-First PWA for Ethiopian Industrial Parks Development Corporation, 2025. https://github.com/yeti2014/IPDC-OSS-platform.

[10] Bhatnagar S. Public Service Delivery: Role of Information and Communication Technology in Improving Governance and Development Impact. Asian Development Bank Economics Working Paper Series, 2014, (391).

[11] Biorn-Hansen A, Majchrzak T A and Gronli T.-M. Progressive Web Apps: The Possible Web-Native Unifier for Mobile Development. In: Proceedings of the International Conference on Web Information Systems and Technologies, 2017: 344 ∼ 351.

[12] Brautigam D, Tang X. African Shenzhen: China's Special Economic Zones in Africa. Journal of Modern African Studies, 2011, 49(1): 27 ∼ 54.

[13] Ed. by Jordan P W, Thomas B, McClelland I L, et al. SUS: A "Quick and Dirty" Usability Scale. In: Usability Evaluation in Industry. Taylor & Francis, 1996: 189 ∼ 194.

[14] Chen K.-M, Nath R. Connectivity Patterns and Digital Service Delivery in Developing Regions. Telecommunications Policy, 2016, 40(10–11): 983 ∼ 1000.

[15] Chen M, Liu W and Lu D. Challenges and the Way Forward in China's New-Type Urbanization. Land Use Policy, 2022, 55: 334 ∼ 339.

[16] Chen Y.-C, Liu J. South–South Technology Transfer: The Case of China–Africa Cooperation. Science and Public Policy, 2018, 45(5): 702 ∼ 714.

[17] Cherukuri S, Patel D. Implementing Offline-First Progressive Web Applications: Patterns for Data Synchronization and Conflict Resolution. IEEE Software, 2024, 41(2): 64 ∼ 72.

[18] Denbu A, Wondimu B. Challenges of E-Government Implementation in Ethiopian Public Organizations. Ethiopian Journal of Public Management, 2019, 6(2): 18 ∼ 39.

[19] Donner J. After Access: Inclusion, Development, and a More Mobile Internet. Cambridge, MA: MIT Press, 2015.

[20] Edgerton D. The Shock of the Old: Technology and Global History Since 1900. Oxford: Oxford University Press, 2007.

[21] El Kadi T H. The Digital Silk Road: China's Technological Influence in Africa. African Affairs, 2024, 123(490): 45 ∼ 67.

[22] Ethio Telecom. Annual Performance Report 2022/2023. 2023.

[23] Fahlander D. Dexie.js: A Minimalistic Wrapper for IndexedDB, 2024. https://dexie.org.

[24] Fall K. A Delay-Tolerant Network Architecture for Challenged Internets. ACM SIGCOMM Computer Communication Review, 2003, 33(4): 27 ∼ 34.

[25] Gaunt M. Service Workers: An Introduction. Google Web Fundamentals, 2020. https://developers.google.com/web/fundamentals/primers/service-workers.

[26] Gebreeyesus M. Industrial Policy and Development in Ethiopia: Evolution and Present Experimentation. Brookings Institution Africa Growth Initiative Working Paper, 2019.

[27] Google. Firebase Documentation: Cloud Firestore Offline Persistence, 2024. https://firebase.google.com/docs/firestore/manage-data/enable-offline.

[28] Google. Workbox: JavaScript Libraries for Progressive Web Apps, 2024. https://developer.chrome.com/docs/workbox.

[29] Gregor S, Hevner A R. Positioning and Presenting Design Science Research for Maximum Impact. MIS Quarterly, 2013, 37(2): 337 ∼ 355.

[30] Guteta D, Jiang X. Evaluating Service Delivery in Ethiopian Industrial Parks: A Multi-Stakeholder Perspective. African Journal of Science, Technology, Innovation and Development, 2023, 15(2): 189 ∼ 204.

[31] Guteta D, Jiang X. Industrial Park Development in Ethiopia: Challenges and Prospects. Sustainability, 2022, 14(6): 3512.

[32] Guteta D, Jiang X. Service Quality and Tenant Satisfaction in Ethiopian Industrial Parks: An Empirical Assessment. Journal of African Business, 2024, 25(1): 112 ∼ 134.

[33] Heeks R. Information and Communication Technology for Development (ICT4D). London: Routledge, 2018.

[34] Hensengerth O. South–South Technology Transfer: Who Benefits? A Critical Analysis of China's Technology Engagement in Africa. Third World Quarterly, 2018, 39(8): 1527 ∼ 1544.

[35] Hevner A R, March S T, Park J, et al. Design Science in Information Systems Research. MIS Quarterly, 2004, 28(1): 75 ∼ 105.

[36] Huang L, Zhang W. Smart Industrial Parks in China: From Space Providers to Service Ecosystems. Journal of Urban Technology, 2023, 30(2): 47 ∼ 69.

[37] Industrial Parks Development Corporation. IPDC Annual Report 2019/2020. Addis Ababa, Ethiopia, 2020.

[38] International Telecommunication Union. Measuring Digital Development: ICT Price Trends 2023. 2023.

[39] Kleppmann M, Wiggins A, Hardenberg P van, et al. Local-First Software: You Own Your Data, in Spite of the Cloud. In: Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!) ACM, 2019: 154 ∼ 178.

[40] Kluiver H de, Gagliardone I. The Belt and Road Initiative and Africa's Digital Future: Opportunities, Risks, and Governance. Journal of International Affairs, 2024, 77(1): 113 ∼ 132.

[41] Layne K, Lee J. Developing Fully Functional E-Government: A Four Stage Model. Government Information Quarterly, 2001, 18(2): 122 ∼ 136.

[42] Lessa L, Negash S and Hailemariam M. E-Government Implementation Challenges in Ethiopia. Electronic Journal of Information Systems in Developing Countries, 2019, 85(5): e12093.

[43] Lewis J R. The System Usability Scale: Past, Present, and Future. International Journal of Human–Computer Interaction, 2018, 34(7): 577 ∼ 590.

[44] Liu W, Dunford M. The Belt and Road Initiative and the Digital Silk Road: Infrastructure and Connectivity. Eurasian Geography and Economics, 2023, 64(1): 1 ∼ 30.

[45] Majchrzak T A, Biorn-Hansen A and Gronli T.-M. Progressive Web Apps: The Definite Approach to Cross-Platform Development? In: Proceedings of the Hawaii International Conference on System Sciences, 2018: 5735 ∼ 5744.

[46] March S T, Smith G F. Design and Natural Science Research on Information Technology. Decision Support Systems, 1995, 15(4): 251 ∼ 266.

[47] Meta Platforms. React: A JavaScript Library for Building User Interfaces, 2024. `https://react.dev`.

[48] Mozilla Developer Network. IndexedDB API – Web APIs, 2024. `https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API`.

[49] Muawwal A, Khan S. Offline-First Application Development for Government Services in Connectivity-Constrained Contexts. International Journal of Electronic Government Research, 2024, 20(1): 1 ∼ 22.

[50] Negesa C, Liu D. Chinese Investment in Ethiopian Industrial Parks: Opportunities and Challenges. Journal of Chinese Economic and Business Studies, 2022, 20(3): 245 ∼ 268.

[51] Nicoara N U. Offline First Web Development. Packt Publishing, 2018.
[52] Nielsen J. Usability Engineering. Boston: Academic Press, 1993.
[53] Oqubay A. Industrial Policy and Late Industrialization in Ethiopia. African Development Review, 2018, 30(S1): 94 ∼ 106.
[54] Oqubay A. Made in Africa: Industrial Policy in Ethiopia. Oxford: Oxford University Press, 2015.
[55] Oqubay A, Lin J Y. China–Africa and an Economic Transformation. Oxford: Oxford University Press, 2019.
[56] PR Newswire. Suzhou Industrial Park Launches 'Marvelous City of SIP' One-Stop Mobile Service Platform, 2024. https://www.prnewswire.com.
[57] Peffers K, Tuunanen T, Rothenberger M A, et al. A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, 2007, 24(3): 45 ∼ 77.
[58] Rogers E M. Diffusion of Innovations. New York: Free Press, 2003.
[59] Sahay S, Sein M K and Urquhart C. Flipping the Context: ICT4D, the Next Grand Challenge for IS Research and Practice. Journal of the Association for Information Systems, 2017, 18(12): 837 ∼ 847.
[60] Schreieck M, Wiesche M and Krcmar H. Digital Platform Adaptation: Strategies for Cross-Context Deployment. Information Systems Journal, 2022, 32(4): 789 ∼ 815.
[61] Schumacher E F. Small Is Beautiful: Economics as if People Mattered. Harper & Row, 1973.
[62] Sein M K, Thapa D, Hatakka M, et al. A Holistic Perspective on the Theoretical Foundations for ICT4D Research. Information Technology for Development, 2019, 25(1): 7 ∼ 38.
[63] Tandel S S, Jamadar A. Impact of Progressive Web Apps on Web App Development. International Journal of Innovative Research in Science, Engineering and Technology, 2018, 7(9): 9439 ∼ 9444.
[64] Tencent. WeChat Work (WeCom): Enterprise Communication and Collaboration Platform, 2024. https://work.weixin.qq.com.
[65] Tesfaw M, Gebrehiwot T. Digital Transformation Readiness in Ethiopian Public Enterprises: Challenges and Pathways. Ethiopian Journal of Science and Technology, 2023, 16(1): 33 ∼ 52.
[66] Toyama K. Geek Heresy: Rescuing Social Change from the Cult of Technology. New York: PublicAffairs, 2015.
[67] United Nations. E-Government Survey 2022: The Future of Digital Government. New York, 2022.
[68] Venable J, Pries-Heje J and Baskerville R. FEDS: A Framework for Evaluation in Design Science Research. European Journal of Information Systems, 2016, 25(1): 77 ∼ 89.
[69] Walsham G. ICT4D Research: Reflections on History and Future Agenda. Information Technology for Development, 2017, 23(1): 18 ∼ 41.

[70] Wang J, Chen T and Zhang Y. Digital Governance Platforms in Chinese Industrial Parks: Architecture and Implementation. Government Information Quarterly, 2022, 39(4): 101742.

[71] Wimmer M A. A European Perspective Towards Online One-Stop Government: The eGOV Project. Electronic Commerce Research and Applications, 2002, 1(1): 92 $\sim$ 103.

[72] World Bank. Ethiopia Digital Foundations Project: Implementation Status Report. Washington, DC, 2022.

[73] Yang R, Li J. One-Stop Digital Services in Chinese Smart Parks: A Multi-Case Study. Electronic Government, an International Journal, 2023, 19(1): 78 $\sim$ 99.

[74] Yang R, Zhou M and Li J. Evolving Smart Park Ecosystems: Digital Integration and Tenant Satisfaction in Chinese Industrial Zones. Technological Forecasting and Social Change, 2025, 200: 123095.

[75] Yu W, Xu C. Developing Smart Cities in China: An Empirical Analysis. International Journal of Public Administration in the Digital Age, 2020, 7(2): 1 $\sim$ 20.

[76] Zeng D Z. The Past, Present, and Future of Special Economic Zones and Their Impact. Journal of International Economic Law, 2021, 24(2): 259 $\sim$ 275.

[77] Zheng Y, Hatakka M, Sahay S, et al. Conceptualizing Development in Information and Communication Technology for Development (ICT4D). Information Technology for Development, 2018, 24(1): 1 $\sim$ 14.

致谢

Above all, I give thanks to the Almighty God, whose grace, clarity of mind, and quiet strength carried me through every difficult moment of this journey and made the completion of this thesis possible.

My deepest gratitude goes to my supervisor, **Prof. Zhang Haining**, whose guidance shaped this research at every stage. The feedback was always direct and constructive, and pushed the work in directions I would not have found on my own.

To the **College of Software at Nankai University**, I am truly grateful for the intellectual environment, outstanding facilities, and the opportunity to pursue advanced studies in software engineering. The courses I took broadened my understanding of distributed systems, artificial intelligence, and modern software architecture — all of which fed directly into this thesis.

I also wish to thank the **Ministry of Commerce of the People's Republic of China (MOFCOM)** for making this educational journey possible through the MOFCOM Scholarship programme. Being selected for this opportunity has been a privilege I do not take lightly.

My sincere appreciation goes to the **Industrial Parks Development Corporation (IPDC)** of Ethiopia for their cooperation during the data collection and requirements gathering phases. The time and candid perspectives offered by IPDC staff, park managers, and tenant firm representatives were invaluable in grounding this work in real operational needs rather than abstract assumptions.

I am glad, too, for the friendships and intellectual exchanges shared with **classmates and colleagues** at Nankai University — Ethiopian, Chinese, and international alike. The conversations we had, in classrooms and over shared meals in Tianjin, broadened my perspective in ways that no textbook can replicate. Those friendships made this chapter of my life genuinely memorable.

To my **family in Ethiopia** — my parents, my siblings, and extended family — no words are quite adequate. Your prayers, your encouragement, and the sacrifices you

made on my behalf have been the bedrock of everything I have worked toward. Distance never diminished the warmth and strength I drew from knowing you were behind me.

I also wish to acknowledge the use of AI-assisted software development tools during the implementation phases of this project. These tools supported coding, debugging, and technical problem-solving in building the platform prototype. All research design, data collection, stakeholder evaluations, analysis, and the writing of this thesis are entirely the work of the author.

Finally, this work is dedicated to everyone striving to close the digital divide — to bring the practical benefits of technology to the communities that need them most but are too often the last to reach them. May this research, however modest its contribution, move that effort a small step forward.

Kebede Yetnayet Berhanu
Nankai University, Tianjin, China
March 2026

个人简历

Personal Information

Name:	Kebede Yetnayet Berhanu
Sex:	Female
Date of Birth:	July 26, 1990
Nationality:	Ethiopian
Place of Birth:	Dire Dawa, Ethiopia
Marital Status:	Unmarried
Mobile (China):	+86 182 2284 7825
Mobile (Ethiopia):	+251 909 789 939
Email:	yetiber2014@gmail.com

Education

2023 – 2025	**Master of Science in Software Engineering** College of Software, Nankai University, Tianjin, China Thesis: *Bridging the Digital Divide: An Offline-First Digital One-Stop Service Localized from China's Smart Park Model for Ethiopia's Industrial Parks* Supervisor: Prof. Zhang Haining Funded by MOFCOM Scholarship (Ministry of Commerce, People's Republic of China)
2008 – 2013	**Bachelor of Science in Electrical and Computer Engineering** Addis Ababa Institute of Technology (AAIT), Addis Ababa University, Addis Ababa, Ethiopia
Secondary	Dire Dawa Comprehensive Secondary School (Grades 11–12); Sabiyan Secondary School (Grades 9–10), Dire Dawa, Ethiopia

Professional Experience

Jun 2022 – Present **Industry Projects Service Specialist**
Industrial Parks Development Corporation (IPDC), Ethiopia
Business and asset valuation projects; consulting on feasibility and suitability studies for industrial park development.

Jul 2015 – Nov 2019 **Instrumentation Engineer**
Ethiopian Sugar Corporation — Omo Kuraz 3 Sugar Project, Ethiopia
Installing, maintaining, troubleshooting, and calibrating electrical, instrumentation and control equipment across extraction, boiler, sugar process, and packing stations; maintaining safe work environments and ensuring all systems are in proper working order.

Dec 2014 – Jul 2015 **Electrical Engineer**
Metal & Engineering Corporation (METEC), Ethiopia

Training and Certifications

Aug – Sep 2023 ITEC Programme on Project Management Training & Certification — Trainers/Promoters Programme (ITEC-PMTC), sponsored by Indian Technical and Economic Cooperation, Government of India (conducted in India)

Feb – Mar 2023 Project Management Training, organized by Industrial Project Service (IPS), Ethiopia

Oct 2022 Asset Valuation Training, Ethiopian Management Institute, Ethiopia

Oct – Nov 2022 Marketing Research Training, Addis Ababa Chamber of Commerce and Sectoral Association, Ethiopia

Feb – Aug 2012 Electrical Engineering Attachment, Akaki Basic Metal Industry (METEC), Ethiopia

May – Jun 2019 Management of Maintenance, Ethiopian Sugar Corporation, Ethiopia

Research Interests

- Offline-first and local-first software architectures
- Progressive Web Application (PWA) development
- Digital governance and e-government platforms
- ICT for Development (ICT4D)
- AI-powered service delivery systems
- Cloud computing and distributed systems

Technical Skills

Languages:	TypeScript, JavaScript, Python, Java, SQL
Frontend:	React, Material-UI, HTML5, CSS3
Backend:	Firebase (Firestore, Auth, Storage), FastAPI, Node.js
Databases:	Firestore, IndexedDB (Dexie.js), MySQL, PostgreSQL
Tools:	Git, Vite, Workbox, Docker, VS Code, LaTeX
AI/ML:	Scikit-learn, TensorFlow, Pandas, NumPy
Methodologies:	Agile, Design Science Research, Project Management

Personal Qualities and Competency

- Ability to work under pressure and effectively in team
- Excellent communication skill
- Exceptional interpersonal and self-regulation skills
- Good computer proficiency
- Sound planning and time management skill
- Problem solving and decision-making skills
- Ability of developing team spirit
- Willingness to learn from others
- Working to achieve organizational goal

Awards and Honors

- MOFCOM Scholarship (Ministry of Commerce, People's Republic of China) for Master's Studies at Nankai University (2023–2025)

Languages

Amharic:	Native
English:	Fluent
Chinese (Mandarin):	Basic / Intermediate

Reference

Name:	Prof. Zhang Haining
Position:	Thesis Supervisor
Institution:	College of Software, Nankai University, Tianjin, China
Email:	zhanghaining@nankai.edu.cn